System Design for AI Applications

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

1 System Design for AI Applications

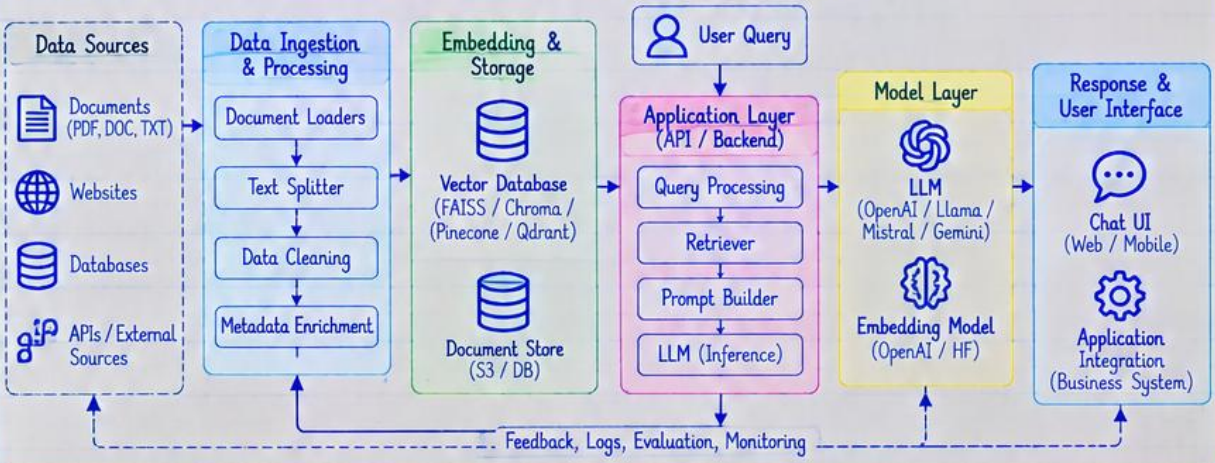
1 What is System Design for AI Applications ?

- System design for AI applications is the process of designing scalable, reliable, secure and cost-effective systems that use AI/ML/LLM models to solve real-world problems.
- It involves data ingestion, model inference, storage, monitoring, and continuous improvement.
- Different from traditional systems because it includes model lifecycle, data pipelines, vector databases, prompt management, evaluation and feedback loops.

2 Key Components

Data Layer	Collect, store, preprocess and manage data (structured, unstructured, streaming).
Model Layer	Training, fine-tuning, inference (LLM/ML models).
Storage Layer	Databases, vector stores, cache, object storage.
Application Layer	APIs, business logic, user interface.
Infrastructure Layer	Compute, scaling, deployment (cloud/on-prem).
Observability Layer	Monitoring, logging, tracing, cost tracking.
Security Layer	Authentication, authorization, data privacy.
Feedback Layer	User feedback, evaluation, continuous improvement.

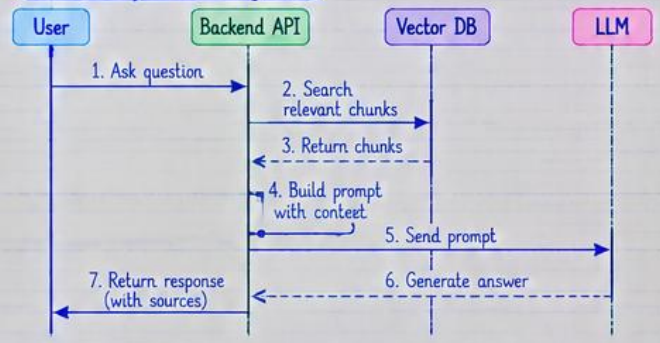
3 High Level Architecture for an AI Application (RAG Example)



4 Detailed RAG Flow

1. Ingest documents from different sources.
2. Split documents into smaller chunks.
3. Generate embeddings and store in vector database.
4. User asks a question.
5. Retrieve relevant chunks using similarity search.
6. Build prompt with context and user question.
7. Send to LLM for generating answer.
8. Return answer with source references.

5 RAG Sequence Diagram



6 Technology Options

Component	Popular Tools / Technologies
Document Loaders	LangChain, LlamaIndex, Unstructured
Text Splitter	RecursiveCharacterTextSplitter, TokenSplitter
Embedding Models	OpenAI, HuggingFace, BGE, E5
Vector Databases	FAISS, Chroma, Pinecone, Qdrant, Weaviate
LLM Models	OpenAI, Llama, Mistral, Gemini, Claude
Backend Framework	FastAPI, Spring Boot, Node.js
Frontend	Streamlit, React, Next.js
Cloud & Deployment	AWS, GCP, Azure, Docker, Kubernetes
Monitoring	Prometheus, Grafana, LangSmith, OpenTelemetry
Others	Redis (cache), Kafka (streaming), S3 (storage)

7 Scalability Considerations

- Use stateless API servers behind a load balancer.
- Scale vector database and LLM inference separately.
- Use caching (Redis) for frequent queries.
- Asynchronous processing for large documents.
- Use message queues (Kafka / RabbitMQ) for
- Auto-scaling with Kubernetes.
- Optimize chunk size and retrieval strategy.
- Use smaller models or quantized models for cost optimization.
- Implement rate limiting and circuit breakers.

8 Security & Privacy

- Authentication & authorization (SSO, OAuth, JWT).
- Data encryption (at rest and in transit).
- Access control for documents.
- PII data masking/redaction.
- Use private LLM / on-prem if required.
- Audit logs and monitoring.
- Follow compliance (GDPR, HIPAA, etc.).
- Secure API keys and secrets (Use Vault / Secrets Manager).

9 Monitoring & Observability

Logs	Metrics	Tracing	Evaluation	Alerts
(Application logs, LLM logs)	(Latency, cost, usage, errors)	(Request flow across services)	(Answer quality, relevance, hallucination)	(Failure, high cost, poor quality)

10 Key Takeaways

- Design modular and scalable architecture.
- Choose right tools for each component.
- Focus on data quality and retrieval strategy.
- Implement monitoring and evaluation.
- Consider security, privacy, and compliance.
- Optimize for cost, latency and user experinece.
- Continuously improve using feedback.

By Rahul Yadav

2. Requirements, Use Cases and High Level Design Considerations for AI Applications

1 Overview

- Understanding requirements is the first step in designing any AI application.
- It helps to define the scope, identify constraints, and choose the right architecture and technologies.
- AI applications have unique requirements like low latency, high accuracy, data privacy, and continuous model updates.
- Clear requirements and use cases ensure a scalable, secure, and cost-effective design.

2 Types of Requirements

Type	Description
Functional Requirements	What the system should do (features and use cases).
Non-Functional Requirements	How well the system should work (performance, scalability, security, etc.).
Business Requirements	Business goals and success metrics (e.g., user adoption, cost, ROI).
AI/ML Specific Requirements	Accuracy, latency, model updates, data quality, explainability, etc.

3 Functional Requirements (Examples)

- Allow users to upload documents.
- Process and extract information from documents.
- Provide accurate and relevant answers to user questions.
- Support multiple file formats (PDF, DOC, TXT, etc.).
- Maintain chat history for each user.
- Integrate with enterprise authentication.
- Support multiple languages.

4 Non-Functional Requirements (Examples)

- Scalability: Handle increasing users and data.
- Low Latency: Provide response within 2–5 seconds.
- High Availability: 99.9% uptime.
- Reliability: Consistent and accurate responses.
- Security: Data encryption, access control, privacy.
- Cost Efficiency: Optimize infrastructure and model usage.
- Maintainability: Easy to update and extend.
- Observability: Logs, metrics, and tracing.

5 AI/ML Specific Requirements

- High accuracy and relevance of responses.
- Support for different data types (text, images, tables).
- Handle large documents and long context.
- Regular model updates and re-indexing.
- Prevent hallucinations and provide source references.
- Support domain-specific knowledge (internal data).
- Explainability and traceability of responses.

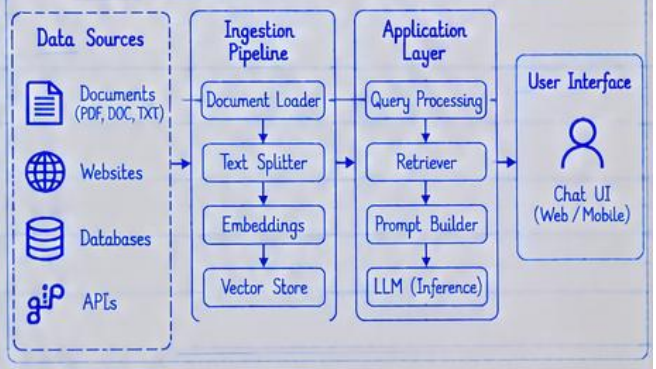
6 Common Use Cases for AI Applications

- Enterprise document Q&A (policies, manuals, reports).
- Customer support chatbot.
- Code assistant for developers.
- Knowledge base search for employees.
- Research and document analysis (legal, finance, healthcare).
- Content summarization and report generation.
- Image and video analysis with AI.
- Multi-modal applications (text + image + audio).

7 High Level Design Considerations

1. Choose the right architecture (RAG, fine-tuning, agents).
2. Identify data sources and ingestion strategy.
3. Select embedding model and vector database.
4. Decide on model (OpenAI, Llama, Claude, etc.).
5. Plan for scalability (stateless services, caching, load balancer).
6. Ensure security and compliance (data privacy, access control).
7. Implement monitoring and evaluation.
8. Optimize for cost, latency, and accuracy.

8 High Level Architecture (RAG Example)



9 Key Metrics to Define Early

Metric	Description
Latency	Time to generate a response.
Throughput	Number of requests per second.
Accuracy	Relevance and correctness of answers.
Cost per Request	Total infrastructure and model cost.
Availability	System uptime (e.g., 99.9%).
Scalability	Ability to handle growing users and data.
User Satisfaction	Feedback score (CSAT).

10 Key Takeaways

- Clearly define functional and non-functional requirements.
- Identify AI/ML specific needs for your use case.
- Choose the right architecture based on requirements.
- Consider scalability, security, cost, and maintainability.
- Define success metrics and monitor them continuously.
- A well-defined requirement analysis helps in building a reliable and production-ready AI application.

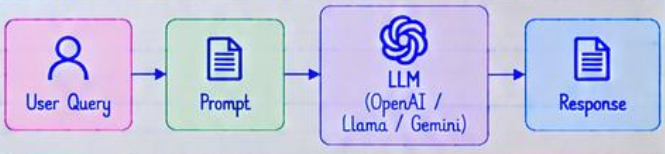
By Rahul Yadav

3. High Level Architecture Patterns for AI Applications

1 Common Architecture Patterns

- Standalone LLM Application (simple use case)
- RAG (Retrieval-Augmented Generation)
- Agentic Architecture (with tools and memory)
- Multi-Agent Architecture (multiple specialized agents)
- Fine-tuned / Custom Model Architecture
- Hybrid Architecture (RAG + Agents + Fine-tuning)
- Enterprise AI Architecture (scalable and secure)

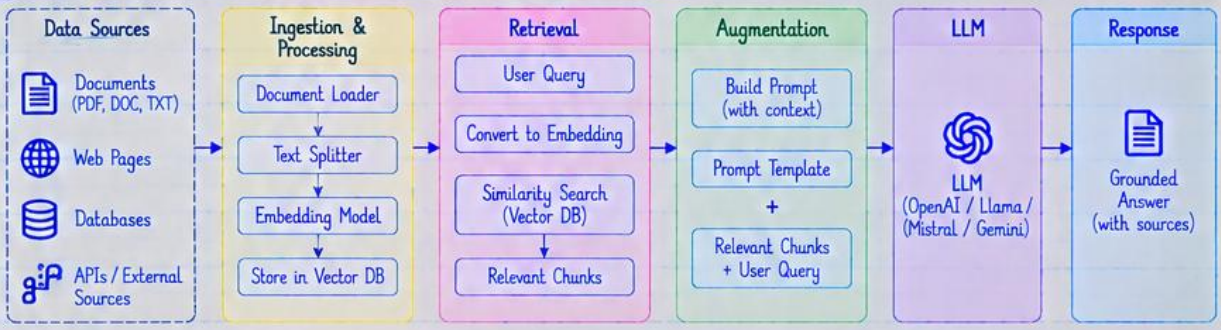
2 Standalone LLM Application



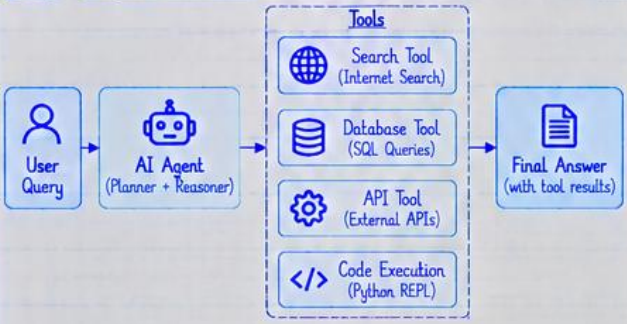
Pros
- Simple architecture
- Quick to build
- Good for general knowledge tasks

Cons
- No access to private data
- May generate incorrect information
- Not suitable for enterprise use cases

3 RAG Architecture (Most Common for Enterprise Use)



4 Agentic Architecture (Using Tools)



Pros
- Can use external tools
- Solves complex multi-step tasks
- More accurate and actionable answers

Cons
- More complex to build
- Need tool integrations
- Possible higher latency and cost

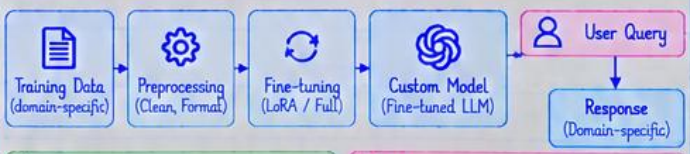
5 Multi-Agent Architecture



Pros
- Handles complex tasks
- Multiple specialized agents
- Scalable and modular

Cons
- More complex design
- Agent coordination needed
- Higher cost and latency

6 Fine-tuned Model Architecture



Pros
- Better domain-specific answers
- More control over model behavior
- No need to provide context every time

Cons
- Requires high-quality training data
- Costly and resource intensive
- Needs model maintenance

7 Hybrid Architecture (RAG + Agents + Fine-tuning)



8 Architecture Comparison

Pattern	Best For	Pros	Cons
Standalone LLM	Simple Q&A	Easy, quick	No private data
RAG	Enterprise knowledge	Accurate, grounded	Needs infrastructure
Agents	Complex tasks	Can use tools	Higher complexity
Multi-Agent	Large workflows	Specialized agents	Harder to manage
Fine-tuned	Domain-specific	Better accuracy	Needs training data
Hybrid	Enterprise use cases	Most powerful	High cost & complex

9 Key Considerations

- Choose the right pattern based on use case.
- Consider latency, cost, scalability, and security.
- Use modular and microservices-based design.
- Plan for future growth and new data sources.
- Evaluate model performance and accuracy.
- Implement monitoring and feedback loops.
- Keep architecture flexible and extensible.

10 Key Takeaways

- There is no one-size-fits-all architecture.
- RAG is most widely used in enterprise.
- Agents enable powerful real-world applications.
- Hybrid architecture gives best results for complex use cases.
- Choose the right pattern based on requirements, data, and business goals.

By Rahul Yadav

4. Data Ingestion and Data Processing Pipeline for AI Applications

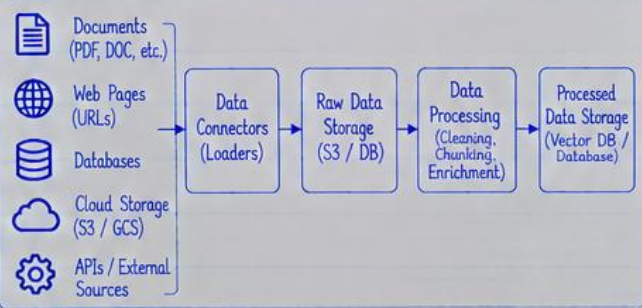
1 Overview

- Data is the foundation of any AI application.
- A well-designed data ingestion and processing pipeline ensures high-quality, relevant, and up-to-date data for model training and inference.
- It handles data from multiple sources, cleans and transforms the data, and stores it in the right format for retrieval or model usage.
- A scalable pipeline is essential to support batch and real-time data.

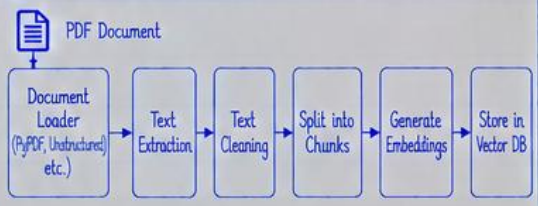
2 Common Data Sources for AI Applications

Source Type	Examples
Documents	PDF, DOC, TXT, PPT, CSV, XLSX
Web	Websites, APIs, Web Pages
Databases	MySQL, PostgreSQL, MongoDB, Oracle
Cloud Storage	S3, GCS, Azure Blob Storage
Enterprise Systems	Confluence, SharePoint, Notion, Jira
Streaming Sources	Kafka, Kinesis, Pub/Sub
Multimedia	Images, Audio, Video
External APIs	Third-party APIs (news, research, etc.)

3 Data Ingestion Pipeline (High Level)



4 Document Ingestion Process (Example)



5 Data Processing Steps

1. Load data from various sources using appropriate loaders.
2. Extract and parse content (text, tables, images, etc.).
3. Clean the data (remove noise, headers, footers, duplicates).
4. Transform the data (normalize, format, convert to plain text).
5. Split the data into smaller chunks.
6. Enrich data with metadata (source, date, title, etc.).
7. Generate embeddings (if using vector search).
8. Store the processed data in a suitable storage (Vector DB / DB).
9. Schedule regular updates for fresh data.

6 Text Splitting Strategies

Strategy	Description	Use Case
Character Splitter	Split text by number of characters.	Simple and fast.
Recursive Splitter	Split using hierarchy (paragraph, sentence, etc.).	Best for long documents.
Token Splitter	Split by token count (using tokenizer).	Useful for LLM context window.
Semantic Splitter	Split based on meaning (using embeddings).	Better context preservation.

7 Metadata Enrichment

- Add useful metadata for better search and filtering.
- Examples: source, file name, title, author, date, category, department, tags, access level, etc.
- Helps in filtering relevant documents during retrieval.
- Can be extracted automatically or manually provided.

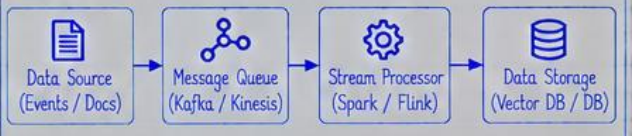
8 Data Storage Options

Storage Type	Purpose	Examples
Vector Database	Store embeddings for semantic search	FAISS, Chroma, Pinecone, Qdrant, Weaviate
Relational Database	Store metadata and structured data	MySQL, PostgreSQL
Cloud Storage	Store original documents	S3, GCS, Azure Blob
Search Engine	Keyword and hybrid search	Elasticsearch, OpenSearch

9 Batch vs Real-time Ingestion

Type	Description	Use Case
Batch Ingestion	Process large amount of data at scheduled intervals.	Initial data load, periodic updates.
Real-time Ingestion	Process data as it arrives using streaming.	Live updates, chat data, logs, real-time documents.

10 Real-time Ingestion Architecture (Example)



11 Tools and Libraries

Component	Tools / Libraries
Document Loaders	LangChain Document Loaders, Unstructured, PyPDF
Text Splitters	RecursiveCharacterTextSplitter, TokenTextSplitter
Embedding Models	OpenAI, HuggingFace, BGE, SentenceTransformer
Vector Databases	FAISS, Chroma, Pinecone, Qdrant, Weaviate
Data Processing	Python (Pandas), Spark, Airflow, dbt
Streaming	Kafka, Kinesis, Pub/Sub
Cloud Storage	AWS S3, GCS, Azure Blob
Orchestration	Apache Airflow, Prefect, Dagster

12 Best Practices

1. Use reliable and scalable data connectors.
2. Clean and preprocess data for better quality.
3. Choose appropriate chunking strategy.
4. Add rich metadata for better retrieval.
5. Use incremental updates to avoid reprocessing all data.
6. Monitor data quality and pipeline health.
7. Handle errors and retries in ingestion process.
8. Ensure data security and access control.

13 Key Takeaways

- A well-designed data pipeline ensures high-quality input for AI models.
- Supports multiple data sources and processing strategies.
- Both batch and real-time ingestion are important depending on use case.
- Metadata and proper storage improve retrieval accuracy.
- Automation, monitoring, and scalability are key for production systems.

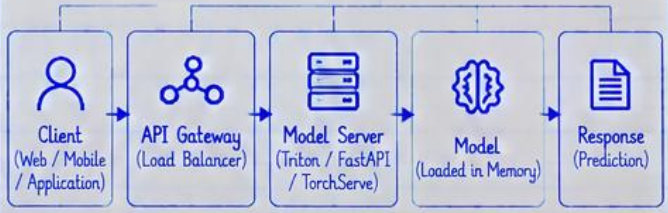
By Rahul Yadav

5. Model Serving and Inference Infrastructure

1 Overview

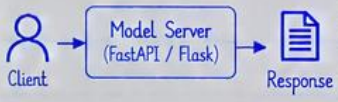
- Model serving makes a trained AI/ML model available for predictions (inference).
- It handles incoming requests, processes them using the model, and returns responses.
- A well-designed serving infrastructure ensures low latency, high throughput, scalability, and reliability.
- Different serving strategies are used based on use case, model size, and traffic pattern.

2 Model Serving Workflow



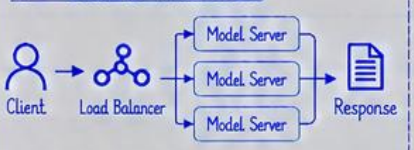
3 Model Serving Architectures

1. Single Model Server



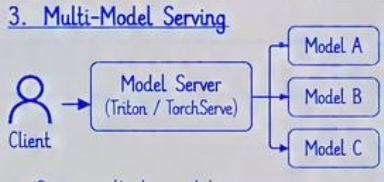
- Simple and easy to deploy.
- Suitable for low to medium traffic.
- Used for small models.

2. 2. Scalable Architecture



- Multiple model server instances.
- Handles high traffic.
- Supports horizontal scaling.
- Used in production systems.

3. Multi-Model Serving



- Serve multiple models in one server.
- Dynamic model loading and unloading.
- Better resource utilization.
- Useful for enterprise use cases.

4 Popular Model Serving Tools

Tool	Description
Triton Inference Server	High performance serving for multiple frameworks (NVIDIA).
FastAPI	Lightweight and easy to build REST APIs for model serving.
TorchServe	Official PyTorch model serving tool.
TensorFlow Serving	Official TensorFlow model serving tool.
BentoML	Model serving and deployment framework.
KServe	Kubernetes native model serving.
Seldon Core	Open source model serving on Kubernetes.
Ray Serve	Scalable model serving for distributed applications.

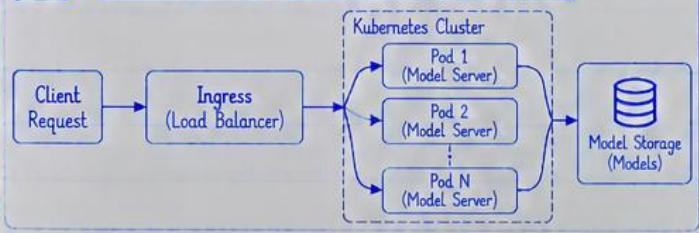
5 Inference Types

Type	Description	Use Case
Real-time Inference	Return prediction in milliseconds.	Chatbots, recommendation systems, fraud detection.
Batch Inference	Process large volume of data asynchornously.	Data labeling, reports, analytics.
Streaming Inference	Process continuous real-time data.	IoT, video/audio processing.
Online Inference	Live requests from users.	Production applications.
Offline Inference	Run periodically on stored data.	Model evaluation, reprocessing.

6 Model Serving Deployment Options

Option	Description
On-Premise	Deploy on own servers, full control, high cost.
Cloud (AWS, GCP, Azure)	Managed services, scalable and flexible.
Kubernetes (K8s)	Containerized deployment with auto-scaling.
Serverless (Lambda, Cloud Run)	Pay per use, good for low traffic.
Edge Deployment	Run models on edge devices for low latency.

7 Kubernetes based Model Serving (Example)



8 Scaling Strategies

1. Horizontal scaling (multiple model server instances).
2. Auto-scaling based on CPU/GPU usage.
3. Use load balancer to distribute requests.
4. Model caching to reduce cold start time.
5. Use optimized model formats (ONNX, TensorRT).
6. Use GPU/TPU for high performance models.

9 Monitoring and Observability

Metric	Description
Latency	Time taken to return response.
Throughput	Number of requests per second.
Error Rate	Failed requests percentage.
Resource Usage	CPU, GPU, Memory usage.
Model Drift	Monitor prediction quality over time.
Logs and Traces	Request logs, input/output, errors.
Alerting	Set alerts for high latency, errors, resource usage.

10 Best Practices

1. Use optimized model formats (ONNX, TensorRT).
2. Implement request batching for better throughput.
3. Use caching for frequent predictions.
4. Monitor latency, error rate, and resource usage.
5. Implement auto-scaling and load balancing.
6. Use versioning for model updates.
7. Ensure security (authentication, rate limiting).
8. Test for different load scenarios.

By Rahul Yadav

6. AI Model Selection, LLM Providers and Model Trade-offs

1 Overview

- Choosing the right AI model is critical for building successful applications.
- Different models have different strengths, costs, capabilities and limitations.
- We need to evaluate models based on use case, performance, cost, data privacy and deployment options.
- This page covers popular LLM providers, model selection criteria and key trade-offs to help you make the right decision.

2 Types of AI Models

Model Type	Description	Examples	Use Case
LLM (Text)	Understands and generates text	GPT, Llama, Claude, Gemini, Mistral	Chat, summarization, Q&A, content generation
Multi-modal	Supports text, image, audio, video	GPT-4V, Gemini, Llama 3.2 Vision	Image understanding, document analysis
Embedding Models	Convert text to vector embeddings	OpenAI text-embedding, BGE, E5, Instructor	Similarity search, RAG, classification
Speech Models	Speech to text or text to speech	Whisper, Azure Speech, Google Speech	Transcription, voice assistants
Vision Models	Understand images	CLIP, DALL-E, Stable Diffusion, Llama Vision	Image analysis, image generation

3 Popular LLM Providers

Provider	Popular Models	Strengths	Deployment
OpenAI	GPT-4o, GPT-4, GPT-3.5	High quality, strong reasoning	Cloud (API)
Anthropic	Claude 3 / 3.5	Safe, long context, good reasoning	Cloud (API)
Google	Gemini 1.5 / 2.0	Multi-modal, large context	Cloud (API)
Meta	Llama 3 / 3.1	Open source, customizable	Self-hosted / Cloud
Mistral AI	Mistral Large, Mixtral	Efficient, high performance	Cloud / Self-hosted
Cohere	Command R+	Enterprise ready, good RAG support	Cloud (API)
AWS	Titan	AWS integration, enterprise support	Cloud (API)
Azure	GPT, Llama, Phi	Enterprise, security, compliance	Cloud (API + Private Link)

4 Model Selection Criteria

1. Use Case: Chat, RAG, summarization, classification, etc.
2. Model Capability: Reasoning, coding, multi-modal support.
3. Context Length: Amount of input the model can handle.
4. Latency: Response time requirements.
5. Cost: Per token cost (input and output).
6. Deployment Option: API (cloud) or self-hosted (on-prem).
7. Data Privacy: Whether data can be sent to third-party.
8. Customization: Support for fine-tuning or adapters (LoRA).
9. Ecosystem & Tools: SDKs, community support, integrations.
10. Reliability: Uptime, rate limits, SLA.

5 Model Comparison (Example)

Model	Context Length	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)	Strengths	Best For
GPT-4o	128K	$2.5	$10	High quality, multi-modal	General use
Claude 3.5	200K	$3	$15	Safe, reasoning	Long documents
Gemini 1.5	1M	$1.25	$5	Large context	Large files, multi-modal
Llama 3.1	128K	Free (self-hosted)	Free	Open source	Custom solutions
Mistral Large	128K	$2	$6	Efficient	Production apps

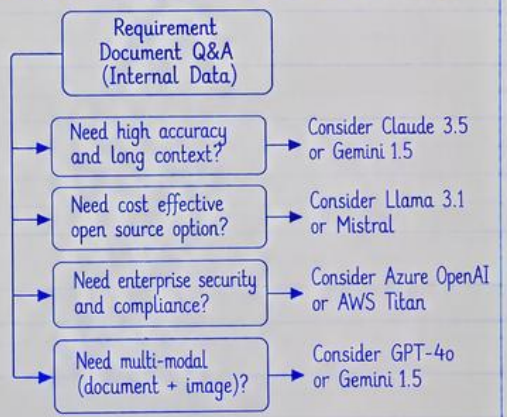
6 Model Selection Workflow



7 Trade-offs

Factor	Option 1	Option 2	Trade-off
Model Size	Small Model (7B)	Large Model (70B+)	Lower cost & latency vs Higher accuracy
Deployment	Cloud API	Self-hosted	Easy to use vs More control & data privacy
Cost	Cheap Models (e.g., GPT-3.5)	Premium Models (e.g., GPT-4, Claude)	Lower cost vs Better quality and reasoning
Latency	Smaller model	Larger model	Faster response vs Better accuracy
Data Privacy	Third-party API	On-prem deployment	Convenience vs Full control
Customization	Prompting	Fine-tuning (LoRA)	Quick and easy vs More control, higher cost

8 Example: Choosing Model for RAG Application



9 Best Practices

1. Clearly define use case and requirements.
2. Compare multiple models before final selection.
3. Test with real data and evaluate quality, latency and cost.
4. Use prompt engineering to improve results.
5. Consider fine-tuning or adapters for domain specific tasks.
6. Monitor usage, cost and performance after deployment.
7. Keep model selection flexible to switch when needed.

10 Key Takeaways

- There is no one-size-fits-all model.
- Choose model based on use case, cost, performance and privacy.
- Evaluate multiple model providers and run POC.
- Consider trade-offs between cost, accuracy, latency and control.
- Use the right deployment option (cloud or self-hosted).
- Keep monitoring and be ready to switch or upgrade models.

By Rahul Yadav

7. Prompt Engineering and RAG Design for AI Applications

1 Overview

- Prompt engineering helps you get better and consistent responses from LLMs.
- RAG (Retrieval-Augmented Generation) combines your private data with LLMs to provide accurate and up-to-date answers.
- A well-designed RAG system improves factuality, reduces hallucinations, and provides domain-specific knowledge.
- This page covers prompt techniques, RAG architecture, and implementation details.

2 Prompt Engineering Techniques

Technique	Description	Example (Use Case)
Zero-shot	Direct question without examples.	Explain microservices.
Few-shot	Provide examples in prompt.	Classify support tickets with examples.
System Prompt	Define behavior and role of model.	You are a helpful banking assistant.
Chain-of-Thought	Ask model to think step by step.	Solve complex reasoning problems.
Role Prompting	Assign a specific role.	You are a loan advisor.
Structured Prompt	Use JSON / specific format.	Return response in JSON.
Context Prompting	Add relevant context.	Use retrieved document as context.

3 Prompt Structure (Template)

System Prompt (Role + Instructions)

```
You are a helpful AI assistant for {domain}.
Follow the instructions and use the context to answer.
If the answer is not in the context, say you don't know.
```

Context (Retrieved Data)

```
{context}
```

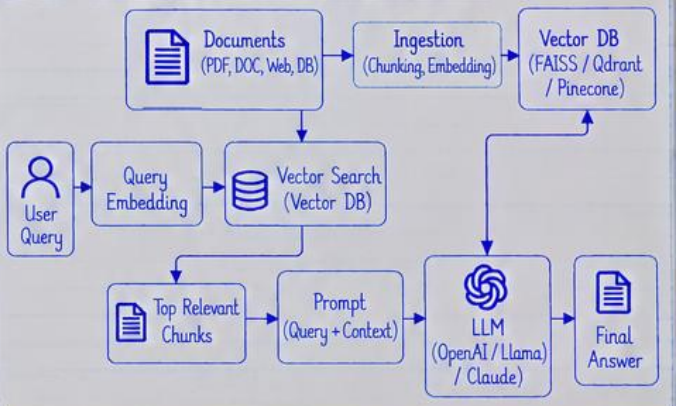
User Query

```
{question}
```

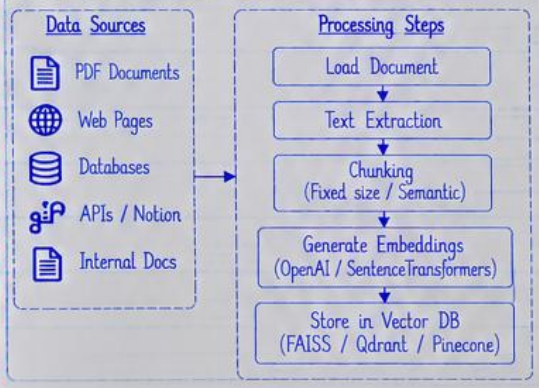
Output Format

```
Return the answer in a clear and concise format.
```

4 RAG Architecture (High Level)



5 Document Ingestion Pipeline



6 Chunking Strategies

Strategy	Description	Use Case
Fixed Size	Split text into fixed number of characters/tokens.	Simple and fast.
Recursive	Split using headings, paragraphs, structure.	Documents with structure (PDF, DOC).
Semantic	Split based on meaning using embeddings.	Better context preservation.
Hybrid	Combination of rules and semantic.	Best for complex documents.

7 Embedding Models (Popular Options)

Model	Dimension	Strengths	Use Case
OpenAI (text-embedding-3-small)	1536	High accuracy	General purpose
OpenAI (text-embedding-3-large)	3072	Better performance	Large documents
SentenceTransformer	384 / 768	Open source	Self-hosted
BGE (BAAI)	768	Good for RAG	Open source, multilingual
Cohere Embed	1024	Good performance	Enterprise use

8 Vector Databases Comparison

Database	Type	Strengths	Use Case
FAISS	Open Source	Fast, simple	Small to medium data
Qdrant	Open Source	Filtering, metadata	Production RAG apps
Pinecone	Managed Cloud	Scalable, easy to use	Enterprise use
Weaviate	Open Source	Graph + Vector	Semantic search
Chroma	Open Source	Simple, lightweight	Prototyping

9 RAG Best Practices

1. Use good chunking strategy (semantic if possible).
2. Use relevant metadata (source, title, page number, etc.).
3. Use re-ranking to improve search results.
4. Provide clear and structured prompts.
5. Set system instructions to reduce hallucinations.
6. Use citation in the response (show source).
7. Handle cases when answer is not in context.
8. Evaluate with real user queries and feedback.

10 Common Use Cases

- Enterprise document Q&A (policies, manuals, reports).
- Customer support chatbot (knowledge base).
- Legal document analysis (contracts, compliance).
- Research assistant (papers, articles).
- Internal company search (confluence, notion, jira).
- Domain-specific AI assistant (banking, healthcare, finance).

11 Key Takeaways

- Prompt engineering and RAG are key to building reliable AI applications.
- Use the right chunking, embedding model, and vector database.
- Provide clear instructions and context to the model.
- Always show source/citations to improve trust.
- Continuously evaluate and improve based on real user feedback.

By Rahul Yadav

8. Vector Databases and Embeddings for AI Applications

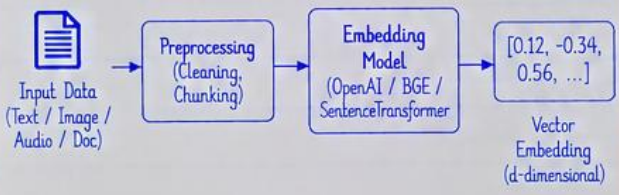
1 Overview

- Vector databases store and search high-dimensional vector embeddings efficiently.
- Embeddings convert text, images, audio, or other data into numerical vectors that capture semantic meaning.
- They are essential for RAG, semantic search, recommendation systems, and similarity matching.
- This page covers embeddings, popular vector databases, architecture, and key considerations.

2 What are Embeddings?

- Embeddings are dense numerical vectors that represent the meaning of data (text, image, etc.).
- Similar data points have similar vector representations.
- Generated using models like OpenAI, SentenceTransformers, BGE, Cohere, etc.
- Usually high-dimensional (e.g., 384, 768, 1536 dimensions).
- Used for semantic search, clustering, classification and retrieval.

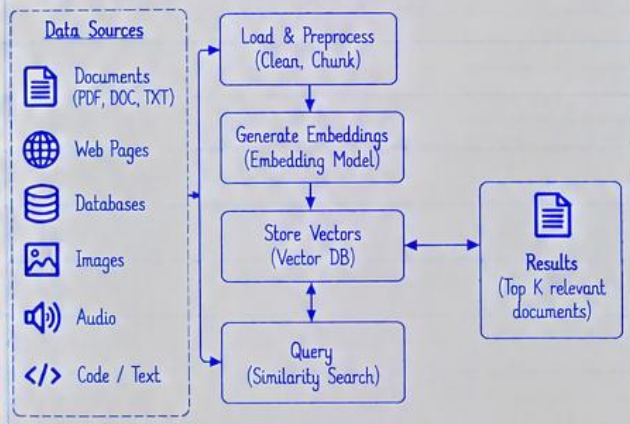
3 Embedding Generation Process



4 Popular Embedding Models

Model	Dimension	Strengths	Use Case
OpenAI (text-embedding-3-small)	1536	High accuracy, multilingual	General purpose
OpenAI (text-embedding-3-large)	3072	Better performance	Complex search
SentenceTransformer (all-MiniLM-L6-v2)	384	Lightweight, fast, open source	Small to medium applications
BGE (BAAI)	768	Good performance	RAG, semantic search
Cohere Embed	1024	Good for enterprise	Multilingual use
Instructor	768	Task-specific	Domain specific tasks

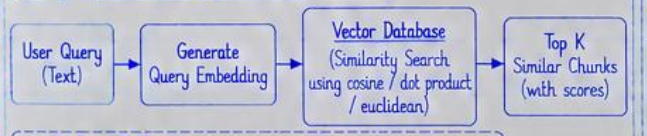
5 Vector Database Architecture



6 Popular Vector Databases Comparison

Database	Type	Strengths	Use Case
FAISS	Open Source (Library)	Fast, simple, good for small/medium datasets	Research, small apps
Qdrant	Open Source	Advanced filtering, payload support	Production RAG apps
Pinecone	Managed (Cloud)	Scalable, easy to use	Enterprise use
Weaviate	Open Source	Hybrid search, Graph + Vector	Semantic search, enterprise use
Milvus	Open Source	Highly scalable, large datasets	Production, big data
Chroma	Open Source	Simple and lightweight	Prototyping, small apps
ElasticSearch (Dense Vector)	Open Source	Hybrid search (text + vector)	Search applications
MongoDB Atlas (Vector Search)	Managed	Integrated with MongoDB	Enterprise use

7 Similarity Search Process



Similarity Metrics:

- Cosine Similarity (most common for text embeddings)
- Dot Product (fast, good for normalized vectors)
- Euclidean Distance (less common for text)

8 Indexing Techniques

Technique	Description	Pros	Cons
Flat Index	Brute force search (exact search)	High accuracy	Slow for large data
HNSW	Graph based index (approximate search)	Fast and accurate	Higher memory usage
IVF (Inverted File)	Cluster based index	Scalable	Lower accuracy than NNSW
PQ (Product Quantization)	Compress vectors	Lower memory usage	Slight drop in accuracy

9 When to Use Vector Database?

1. Semantic search in documents, knowledge base, or chatbots.
2. RAG (Retrieval-Augmented Generation) applications.
3. Recommendation systems (products, content, etc.).
4. Image, audio or video similarity search.
5. Clustering and anomaly detection.
6. Any application where meaning and similarity matters instead of exact keyword match.

10 Best Practices

1. Choose the right embedding model for your use case.
2. Use proper chunking strategy (size and overlap).
3. Clean and normalize data before embedding.
4. Use metadata/payload for filtering (e.g., source, title, date).
5. Use hybrid search (keyword + vector) for better results.
6. Evaluate different similarity metrics and index types.
7. Monitor performance (latency, recall, storage).
8. Keep vectors updated when source data changes.

11 Key Takeaways

- Embeddings convert data into numerical vectors.
- Vector databases enable efficient similarity search.
- Choose the right embedding model, vector database and index type.
- Use metadata and hybrid search for better results.
- Vector databases are a core component of modern AI applications like RAG.

By Rahul Yadav

11. Model Deployment, Scaling and Monitoring

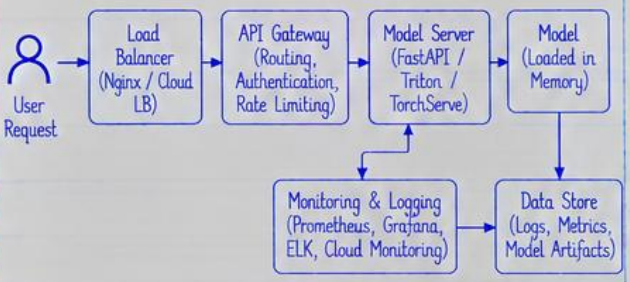
1 Overview

- After building and testing an AI model, we need to deploy it in a production environment.
- Model deployment makes the model available to real users through an API or service.
- We also need to handle scaling, monitoring, logging and updates to ensure reliability.
- This page covers different deployment options, scaling strategies, monitoring techniques and best practices for AI applications.

2 Model Deployment Options

Option	Description	Use Case	Pros	Cons
Local Server	Run on a local machine or VM	Development, small apps	Easy setup, low cost	Not scalable, not reliable
Cloud Virtual Machine (VM)	Deploy on cloud VM (AWS EC2, GCP, Azure)	Medium traffic applications	More control, flexible	Manual scaling, maintenance
Container (Docker)	Package model in a container	Most common approach	Portable, consistent environment	Need container management
Kubernetes	Orchestrate multiple containers	High traffic applications	Auto scaling, high availability	Complex setup
Serverless (AWS Lambda, Cloud Functions)	Run model as serverless function	Lightweight inference, low traffic	No server management, pay per use	Limited execution time, cost for high traffic

3 Model Deployment Architecture



4 Model Serving Frameworks

Framework	Description	Use Case	Strengths
FastAPI	Python web framework	Custom model API	Simple, fast, flexible
Triton Inference Server	NVIDIA open source server	High performance inference	Supports multiple frameworks, GPU optimization
TorchServe	PyTorch model serving	PyTorch models in production	Easy deployment, model management
TensorFlow Serving	TensorFlow model serving	TensorFlow models	Scalable, high performance
BentoML	Model serving and packaging	End to end model deployment	Simple, flexible, production ready

5 Containerization with Docker

1. Create a Dockerfile with required dependencies.
2. Copy your model and code into the container.
3. Build Docker image.
4. Run container locally.
5. Deploy to cloud or Kubernetes.

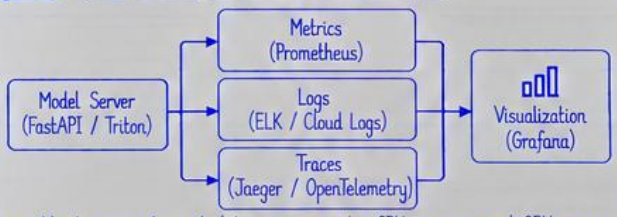
Example Dockerfile (Python)

```
FROM python:3.10
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host",
     "0.0.0.0", "--port", "8000"]
```

6 Scaling Strategies

Strategy	Description	Use Case	Tools
Horizontal Scaling	Increase number of instances	High traffic	Kubernetes, ECS, Auto Scaling
Vertical Scaling	Increase CPU / Memory	Medium traffic	Cloud VM, Larger instance
Auto Scaling	Automatically scale based on metrics	Variable traffic	Kubernetes HPA, Cloud Auto Scaling
Load Balancing	Distribute requests across instances	High availability	Nginx, AWS ALB, GCP Load Balancer
Caching	Cache responses to reduce load	Repeated requests	Redis, CDN

7 Monitoring and Observability



- Monitor request count, latency, error rate, CPU, memory and GPU usage.
- Track model performance (accuracy, drift, output distribution).
- Set up alerts for high latency, errors or resource usage.
- Use logging and tracing to debug issues.
- Tools: Prometheus, Grafana, ELK Stack, Cloud Monitoring, OpenTelemetry.

8 Model Lifecycle Management



- Use model versioning to track all versions.
- A/B testing or canary deployment for new models.
- Monitor model drift and performance degradation.
- Automate retraining pipeline (MLOps).
- Tools: MLflow, DVC, Kubeflow, SageMaker, Vertex AI.

9 Common Challenges

1. High latency for large models.
2. Scaling for variable traffic.
3. Managing model versions.
4. Handling model drift over time.
5. Ensuring security and data privacy.
6. Monitoring model performance in production.
7. High infrastructure cost.

10 Best Practices

1. Use containers (Docker) for consistent deployment.
2. Use Kubernetes for scaling and high availability.
3. Implement monitoring and alerting.
4. Optimize model for latency and resource usage (quantization, batching).
5. Use caching for repeated requests.
6. Maintain proper logging and tracing.
7. Automate deployment and retraining pipelines.
8. Ensure security, authentication and rate limiting.
9. Continuously monitor and improve based on real user data.

11 Key Takeaways

- Model deployment is an essential step to make AI applications available to real users.
- Use the right deployment option based on your use case and traffic.
- Implement monitoring, scaling and versioning for reliability.
- Follow best practices to ensure performance, security and cost efficiency.

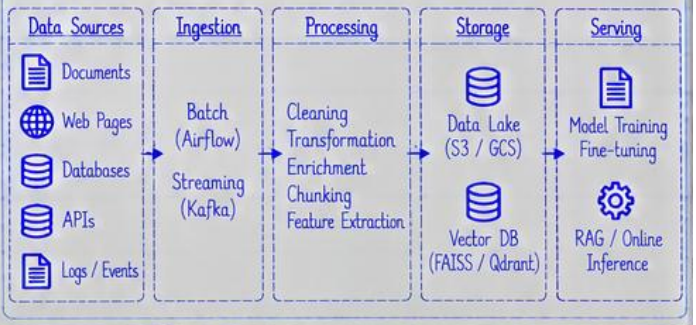
By Rahul Yadav

10. Data Pipeline and Data Management for AI Applications

1 Overview

- High quality data is the foundation of any AI application.
- A well-designed data pipeline collects, stores, processes and serves data for training and inference.
- This page covers end-to-end data pipelines, data storage options, data processing, feature engineering, data versioning and data governance for AI applications.

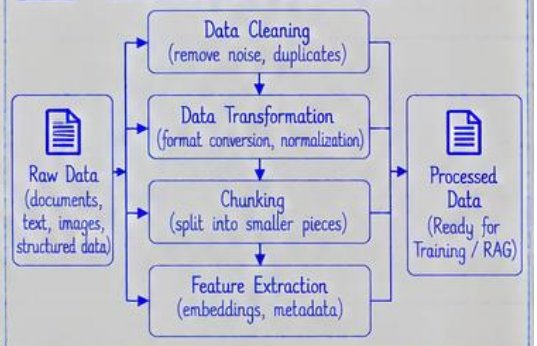
2 End-to-End Data Pipeline Architecture



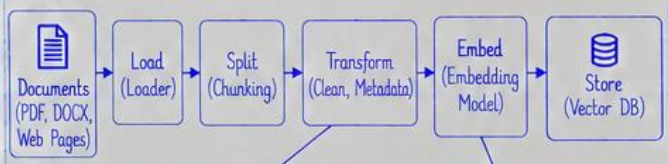
3 Data Ingestion Methods

Method	Description	Use Case	Tools
Batch Ingestion	Periodic data load (from files, DB)	Large datasets, training data	Airflow, Spark, AWS Glue
Streaming Ingestion	Real-time data processing	Real-time apps, logs, events	Kafka, Kinesis, Pulsar
API Ingestion	Fetch data from external APIs	Third-party data, web data	Python, Airflow
File Ingestion	Read files (PDF, CSV, TXT, etc.)	Documents, reports	Python, LangChain, Unstructured

4 Data Processing Pipeline



5 Document Processing for RAG



Example Chunking Strategies

- Fixed size chunks (e.g. 500 tokens)
- Recursive character split
- Semantic chunking (based on meaning)
- Document structure based (headings, paragraphs)

Metadata Examples

- Source (file name, URL)
- Page number
- Document type
- Date, author, tags

6 Data Storage Options

Storage	Type	Strengths	Use Case
S3 / GCS	Object Storage	Scalable, low cost	Store raw and processed data
PostgreSQL	Relational DB	Structured data, ACID compliance	Application data, metadata
MongoDB	NoSQL DB	Flexible schema	Logs, unstructured data
FAISS	Vector DB	Fast, simple	Small to medium embeddings
Qdrant	Vector DB	Scalable, filtering	Production RAG apps
Pinecone	Managed Vector DB	Fully managed	Enterprise use
Weaviate	Vector DB	Graph + Vector	Semantic search
Redis	In-memory	Fast, caching	Query cache, session data

7 Feature Engineering (for ML/Analytics)

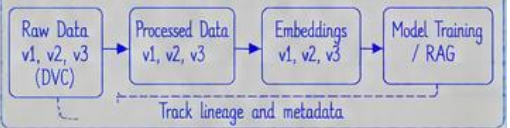
- Convert raw data into meaningful features.
- Examples: text preprocessing, TF-IDF, embeddings, image features, numerical features.
- Good features improve model performance.
- For LLM applications, embeddings are the most common feature representation.

Feature Engineering Steps

1. Understand data
2. Clean and preprocess
3. Generate features (embeddings)
4. Store with metadata
5. Validate and iterate

8 Data Versioning and Lineage

- Track different versions of data and embeddings.
- Helps in reproducibility and debugging.
- Useful when data keeps updating.
- Tools: DVC, MLflow, LakeFS, Delta Lake.



9 Data Governance and Quality

- Ensure data privacy, security and compliance.
- Maintain data quality (no duplicates, correct format).
- Handle sensitive data and PII (masking/encryption).
- Implement access control and auditing.
- Follow regulations (GDPR, HIPAA, etc.).

Key Data Quality Checks

- Remove duplicates
- Validate format
- Check missing values
- Detect and remove noise
- Verify metadata

10 Best Practices

1. Use automated data pipelines (Airflow, Kafka).
2. Maintain clean and well-structured data.
3. Use proper chunking and metadata.
4. Choose the right storage for your use case.
5. Implement data versioning and lineage.
6. Ensure data security and compliance.
7. Monitor data quality and pipeline health.
8. Keep your pipeline scalable and cost-efficient.

11 Key Takeaways

- Data pipeline is critical for AI applications.
- Choose the right ingestion, storage and processing strategy.
- High quality data leads to better model performance.
- Use versioning, governance and monitoring for reliable systems.
- A well-designed data pipeline supports both training and real-time inference.

By Rahul Yadav

9. LLM Application Architectures and Design Patterns

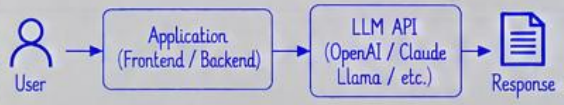
1 Overview

- LLM applications can be designed using different architectures and design patterns depending on the use case.
- A good architecture ensures scalability, reliability, cost efficiency, and better user experience.
- This page covers common LLM application architectures and important design patterns used in real-world systems.

2 Common LLM Application Architectures

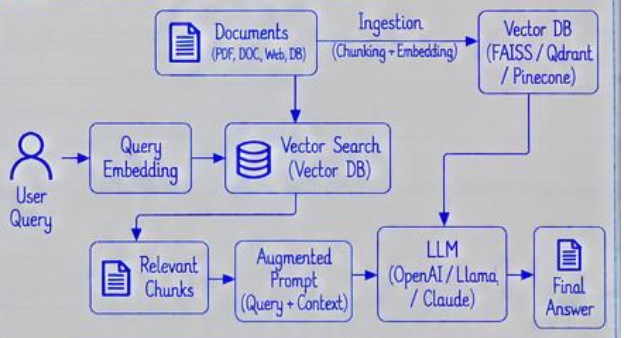
Architecture	Description	Use Case
Basic LLM App	Directly call LLM via API and return response.	Simple Q&A, chatbots.
RAG Architecture	Combine user query with retrieved context.	Document Q&A, knowledge base.
Multi-Agent Architecture	Multiple agents with specialized roles.	Complex workflows, automation.
Tool-Integrated App (Agentic)	LLM can call external tools and APIs.	Perform actions, real-time data.
Fine-tuned Model App	Use domain specific fine-tuned model.	High accuracy, private and domain use cases.

3 Basic LLM Application Architecture

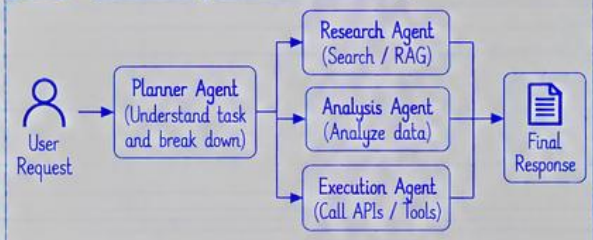


1. User enters a prompt.
2. Application sends the prompt to LLM API.
3. LLM processes and returns a response.
4. Application displays the response to the user.

4 RAG Architecture (Detailed)

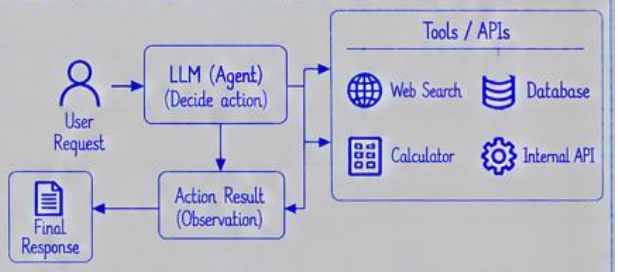


5 Multi-Agent Architecture



- Multiple agents work together to solve complex tasks.
- Each agent has a specific role and skill.
- Useful for research, automation, and enterprise workflows.

6 Tool-Integrated (Agentic) Architecture

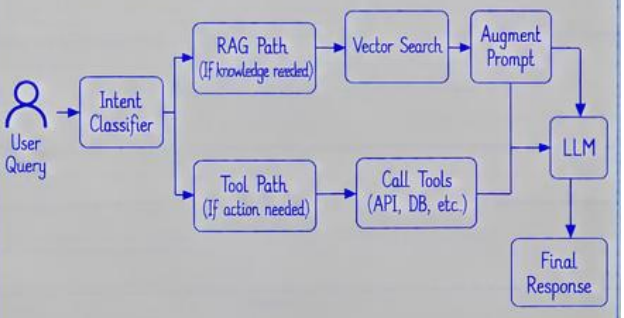


- LLM decides which tool to use based on the user request.
- Can call multiple tools (search, DB, API, etc.).
- Useful for real-time data, automation, and task execution.

7 Common Design Patterns

Pattern	Description	Use Case
Prompt Template	Use predefined template with variables.	Consistent and structured prompts.
RAG Pattern	Retrieve relevant context and augment the prompt.	Document Q&A.
Tool Calling Pattern	Allow LLM to call external tools/APIs.	Real-time data, actions.
Memory Pattern	Maintain conversation history and context.	Chatbots, assistants.
Agent Pattern	Use multiple agents for complex tasks.	Research, automation.
Fallback Pattern	Use alternative model or logic on failure.	Improve reliability.
Streaming Pattern	Stream tokens to user in real-time.	Better user experience.

8 Example: RAG with Tools (Hybrid Pattern)



9 Important Considerations

1. Choose the right architecture based on the use case.
2. Consider latency, cost, and scalability.
3. Handle large context using RAG or memory.
4. Use tools and agents for real-world actions.
5. Implement proper error handling and fallbacks.
6. Monitor performance, logs, and user feedback.
7. Ensure security and data privacy.

10 Key Takeaways

- There is no one-size-fits-all architecture.
- RAG, multi-agent, and tool-integrated patterns are widely used.
- Good design patterns improve reliability, scalability, and user experience.
- Choose the right model, tools, and vector database for your use case.
- Always monitor, evaluate, and iterate based on real usage.

By Rahul Yadav

12. Security, Privacy and Governance in AI Applications

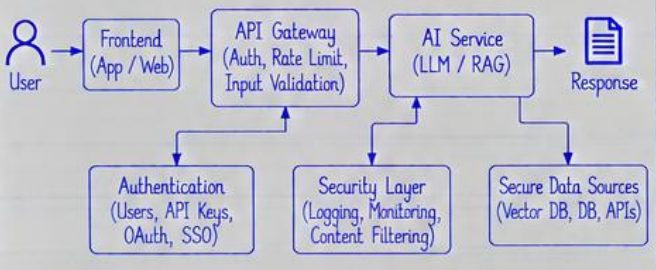
1 Overview

- Security, privacy and governance are critical for building reliable and responsible AI systems.
- AI applications often handle sensitive data, proprietary information and user queries.
- This page covers common risks, security techniques, privacy methods, governance practices and compliance standards for AI applications.
- Following these practices helps prevent data leaks, misuse, prompt injection and ensures ethical AI usage.

2 Common Security Risks in AI Applications

Risk	Description	Example	Impact
Prompt Injection	Manipulate model with crafted input	Ignore previous instructions	Data leakage, wrong output
Data Leakage	Sensitive information exposed in responses	PII, confidential documents	Privacy breach
Insecure Plugins / Tools	Untrusted external tools or APIs	Malicious API call	Unauthorized access
Model Jailbreak	Bypass safety restrictions	Generate harmful content	Misuse, policy violation
Adversarial Input	Input designed to fool the model	Hidden instructions	Incorrect output
Data Poisoning	Manipulate training data	Add fake or biased data	Biased or incorrect model behavior

3 Secure AI Application Architecture



4 Authentication and Access Control

- Use proper authentication (API Keys, OAuth2, JWT).
- Implement role-based access control (RBAC).
- Restrict access to sensitive data and tools.
- Use separate environments for dev, staging and prod.
- Rotate API keys and secrets regularly.
- Use least privilege principle (only required permissions).

Access Control Flow

User Request → Authenticate (Validate Token) → Check Role and Permissions → Allow or Deny

5 Input and Output Security

- Validate and sanitize user input.
- Use content filtering to detect harmful or malicious input.
- Prevent prompt injection using safeguards.
- Restrict model from performing unauthorized actions.
- Filter model output for sensitive information (PII, confidential data).
- Use guardrails and moderation APIs (OpenAI, Azure, etc.).

Input Validation Techniques

- Remove malicious patterns
- Limit input length
- Use allowlist / blocklist
- Detect injection attempts
- Validate file types (PDF, DOC, etc.)

Output Filtering Techniques

- Detect and mask PII (name, email, phone)
- Remove unsafe or harmful content
- Use LLM guardrails / moderation APIs
- Validate structured output (JSON schema)
- Log and monitor responses

6 Data Privacy Techniques

- Avoid sending sensitive data directly to public LLMs.
- Anonymize or mask PII before processing.
- Use data encryption (in transit and at rest).
- Follow data minimization (only necessary data).
- Use private deployment for highly sensitive data.
- Implement retention policies (delete data when not needed).

PII Masking Example

Input : My name is Rahul Yadav and my phone number is 9876543210.

↓

Masked Output : My name is [NAME] and my phone number is [PHONE].

7 Secure Data Handling

Area	Best Practice	Use Case
Data at Rest	Encrypt using AES-256	Vector DB, databases, files
Data in Transit	Use HTTPS / TLS	API calls, data transfer
Secrets Management	Use vaults (AWS Secrets Manager, HashiCorp Vault)	API keys, DB credentials
Access Logs	Log and audit access	User activity tracking
Data Isolation	Use separate environments	Dev / Staging / Production
Backup & Recovery	Regular backups and encryption	Disaster recovery
Data Retention	Define retention period	Compliance and privacy

8 Compliance and Governance

Standard / Framework	Description
GDPR	Data privacy regulation in EU
HIPAA	Healthcare data privacy in US
ISO 27001	Information security management
SOC 2	Security, availability and confidentiality
NIST AI RMF	AI risk management framework
Responsible AI	Fairness, transparency and ethical AI use
Company Policies	Internal data handling and AI usage guidelines

9 Monitoring and Auditing

- Monitor user requests, responses and system usage.
- Track unusual patterns (suspicious inputs, high usage).
- Log access to sensitive data and tools.
- Use monitoring tools (Prometheus, Grafana, ELK).
- Set alerts for security incidents.
- Regularly audit logs for compliance.

Logs (Requests, Responses) → Monitoring (ELK / Grafana) → Alerts (Email / Slack) → Action (Investigate)

10 Best Practices

1. Follow security by design.
2. Implement authentication and authorization.
3. Validate and filter inputs and outputs.
4. Protect sensitive data and use encryption.
5. Use guardrails and content moderation.
6. Follow compliance and governance standards.
7. Monitor, log and audit AI application usage.
8. Regularly update dependencies and security patches.
9. Perform risk assessment for AI use cases.
10. Educate users about safe and responsible AI usage.

11 Key Takeaways

- Security, privacy and governance are essential for building trustworthy AI applications.
- Use authentication, input/output filtering, encryption and monitoring.
- Follow compliance standards and responsible AI practices.
- Continuously evaluate and improve security measures based on real usage and new risks.

By Rahul Yadav

13. AI Evaluation, Testing, and Quality Assurance

1 Overview

- Evaluation and testing are essential to ensure AI applications are accurate, reliable, safe and aligned with user expectations.
- Unlike traditional software, AI models produce probabilistic outputs, so we need specialized metrics and testing methods.
- This page covers evaluation types, metrics, testing strategies, popular tools and best practices for AI applications (LLM, RAG, ML).

2 Why Evaluation is Important?

1. Ensures model gives correct and relevant responses.
2. Helps detect bias, hallucinations and harmful outputs.
3. Improves user trust and reliability.
4. Validates model performance for real-world use cases.
5. Helps compare different models and approaches.
6. Ensures compliance with safety and governance standards.

3 Types of Evaluation

Type	Description	Use Case
Automatic Evaluation	Uses metrics to evaluate model output automatically.	Large scale testing, CI/CD.
Human Evaluation	Humans review and rate the output.	Quality, relevance, safety.
Offline Evaluation	Evaluate on predefined dataset.	Before deployment.
Online Evaluation	Evaluate with real user traffic (A/B testing).	After deployment.
Task Specific Evaluation	Use custom metrics for specific tasks.	RAG, summarization, classification, code gen.
Safety & Bias Evaluation	Check harmful, biased or toxic outputs.	Production readiness.

4 Common Evaluation Metrics

Task Type	Metric	Description
Text Generation	BLEU, ROUGE, METEOR	Measures similarity with reference text.
LLM / RAG	Relevance, Faithfulness, Context Precision/Recall	Evaluates accuracy and groundedness.
Classification	Accuracy, Precision, Recall, F1 Score	Measures classification performance.
Summarization	ROUGE, BERTScore	Measures summary quality.
Code Generation	Pass@k, BLEU, Exact Match	Checks correctness of generated code.
Embedding Models	Cosine Similarity, Recall@K	Evaluates similarity search performance.
Safety Evaluation	Toxicity Score, Bias Score	Detects harmful or biased outputs.

5 Evaluation Process for LLM / RAG

Test Dataset (Questions + References) → Generate Model Responses → Evaluate (Automatic + Human) → Analyze Results (Metrics, Errors) → Improve (Model, Prompt, Data, Pipeline)

RAG Evaluation Metrics

- Answer Relevance (Is the answer relevant?)
- Faithfulness (Is the answer grounded in retrieved context?)
- Context Precision (Are retrieved chunks relevant?)
- Context Recall (Are all relevant chunks retrieved?)
- Hallucination Rate (Non-grounded information)
- User Satisfaction (Human evaluation)

6 Testing Strategies

1. Unit Testing – Test individual components (retriever, parser, API).
2. Dataset Testing – Use benchmark datasets for evaluation.
3. Adversarial Testing – Test with tricky or malicious inputs.
4. Bias & Fairness Testing – Check for biased or toxic outputs.
5. Regression Testing – Ensure new model version doesn't degrade.
6. A/B Testing – Compare two models or prompts in production.
7. Load Testing – Test performance under high traffic.
8. Red Teaming – Try to break the model with harmful inputs.

7 Example: RAG Evaluation Workflow

Question (Query) → Retrieve Top K Chunks (Vector DB) → Generate Answer (LLM) → Evaluate (Relevance, Faithfulness, Context Recall) → Feedback (Logs & Human Review)

8 Popular Evaluation Tools

Tool	Use Case	Features
RAGAS	RAG evaluation	Relevance, faithfulness, context precision/recall
LangChain Eval	LLM & RAG evaluation	Custom evaluators, datasets
DeepEval	LLM testing	Bias, toxicity, hallucination tests
TruLens	Observability & evaluation	Feedback, tracing, metrics
PromptFoo	Prompt testing	A/B testing, regression tests
Helm	LLM evaluation (Stanford)	Model comparison, scenarios
OpenAI Evals	LLM evaluation	Custom eval datasets
MLflow	Experiment tracking	Track models, metrics, versions
Weights & Biases	Experiment tracking	Metrics, logs, comparisons

9 Human Evaluation Guidelines

1. Use clear evaluation criteria (relevance, accuracy, helpfulness).
2. Use rating scale (e.g., 1 to 5).
3. Evaluate with multiple reviewers to reduce bias.
4. Provide reference answers where possible.
5. Check for harmful, biased or inappropriate content.
6. Collect qualitative feedback for improvement.

10 Common Challenges

- No single metric works for all use cases.
- Human evaluation can be costly and time-consuming.
- Handling subjective answers (multiple valid outputs).
- Detecting hallucinations and biased content.
- Maintaining evaluation datasets over time.
- Performance may vary across domains and languages.

11 Best Practices

1. Use a combination of automatic and human evaluation.
2. Define clear evaluation criteria and success metrics.
3. Test for accuracy, safety, bias and robustness.
4. Use real-world datasets and user feedback.
5. Continuously monitor model performance after deployment.
6. Maintain evaluation datasets and update regularly.
7. Use observability tools to track inputs, outputs and feedback.
8. Iterate and improve based on evaluation results.

12 Key Takeaways

- Evaluation and testing are critical for building reliable and trustworthy AI applications.
- Use the right metrics and tools based on your use case (LLM, RAG, ML).
- Combine automatic and human evaluation for better results.
- Continuously monitor, test and improve your model in production.

By Rahul Yadav

14. AI Agents, Tool Calling and Multi-Agent Systems

1 Overview

AI agents are LLM-powered systems that can autonomously plan, reason, use tools and take actions to achieve a goal.

- Tool calling enables agents to interact with external tools such as APIs, databases, search engines, files, etc.
- Multi-agent systems involve multiple specialized agents working together to solve complex tasks.
- This page covers architecture, components, tool calling flow, popular frameworks, design patterns and best practices for building AI agents.

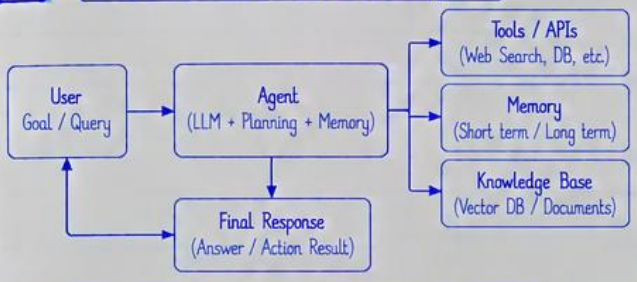
2 What is an AI Agent?

- An AI agent uses an LLM to understand a goal, plan steps, call tools, observe results and make decisions.
- It can work autonomously or with human supervision.
- Agents are useful for research, automation, data analysis, customer support, code generation and more.

Key Characteristics

1. Goal oriented
2. Can plan and reason
3. Can take actions
4. Can use external tools
5. Remembers past interactions (memory)
6. Can work autonomously or with humans

3 Basic Architecture of an AI Agent



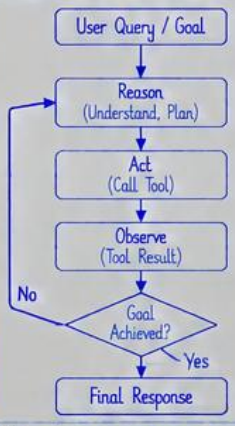
4 Tool Calling Workflow

1. User gives a task or query.
2. Agent analyzes the query and decides which tool to use.
3. Agent generates tool call with required parameters.
4. Tool executes and returns the result.
5. Agent observes the result, updates its reasoning and decides next step (call another tool or give final answer).
6. Final response is generated for the user.

5 Types of Tools for Agents

Tool Type	Description	Examples	Use Case
Web Search	Search real-time information	Google Search, SerpAPI	Get latest information
Database	Query structured data	MySQL, PostgreSQL, MongoDB	Business data, reports
API Tools	Call external APIs	REST APIs, GraphQL	Integrations, external systems
File Tools	Read / write files	Local files, S3	Document processing
Code Execution	Run code / scripts	Python REPL, Sandbox	Data analysis, automation
Domain Tools	Custom tools for specific tasks	Calculator, Email, CRM	Business workflows

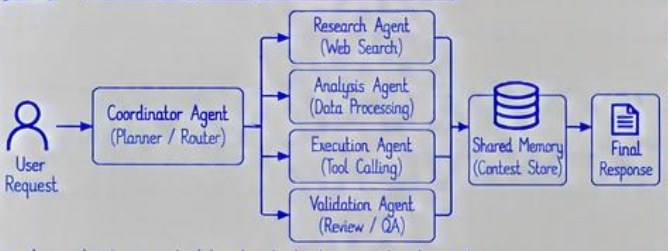
6 Agentic Workflow (ReAct Pattern)



ReAct Steps

1. Reason - LLM decides what to do.
2. Act - Call the tool.
3. Observe - Get tool result and analyze.
4. Repeat - Continue until task is completed.

7 Multi-Agent System Architecture



- A coordinator agent delegates tasks to specialized agents.
- Each agent has a specific role and skill.
- Agents can share context using a common memory or message bus.
- Useful for complex workflows like research, report generation, automation, etc.

8 Popular Frameworks for Building Agents

Framework	Description	Key Features	Use Case
LangChain	Framework for LLM apps and agents	Tools, memory, Chains, agents	General purpose agent development
LlamaIndex	Data + agent framework	Data ingestion, tool integration	RAG agents, knowledge agents
AutoGen	Multi-agent framework (Microsoft)	Conversable agents, group chat	Multi-agent systems, research, automation
CrewAI	Simple multi-agent framework	Role based agents, tasks, crew	Collaborative agents
Semantic Kernel	Microsoft framework	Tool calling, plugins, memory	Enterprise agents
OpenAI Assistants	Built-in agent tools	Function calling, file search, code	Production ready assistants

9 Use Cases

1. Research and information gathering
2. Data analysis and reporting
3. Code generation and execution
4. Business process automation
5. Customer support and chatbots
6. Document analysis and summarization
7. Multi-agent problem solving (e.g., research + analysis + report)
8. Integration with external systems (CRM, ERP, Email, etc.)

10 Best Practices

1. Define clear goals and agent roles.
2. Use the right tools for each task.
3. Maintain conversation memory.
4. Handle tool failures and retries.
5. Add guardrails and validation.
6. Use structured outputs (JSON).
7. Monitor logs, tool usage and performance.
8. Keep agents focused and avoid infinite loops.
9. Test with real-world scenarios.
10. Ensure security and access control for tools.

11 Common Challenges

1. Hallucinations and incorrect tool usage.
2. Infinite loops or repeated actions.
3. Managing context and memory.
4. Tool failures and API errors.
5. Coordination between multiple agents.
6. Cost and latency for complex workflows.
7. Security risks (unauthorized tool access).
8. Ensuring reliable and consistent outputs.

12 Key Takeaways

- AI agents can plan, reason, use tools and take actions to accomplish complex tasks.
- Tool calling enables agents to interact with external systems.
- Multi-agent systems improve capability by using specialized agents.
- Use the right framework, tools and best practices to build reliable and secure agents.

By Rahul Yadav

15. AI Application Performance Optimization

Latency, Caching, Throughput, and Cost

1 Overview

- Performance optimization is important to make AI applications fast, reliable, scalable and cost-effective.
- Key factors: latency, caching, throughput and cost.
- This page covers techniques, architecture patterns, caching strategies, batching, concurrency, cost optimization and best practices.
- These optimizations help in providing better user experience and efficient usage of AI resources.

2 Key Performance Metrics

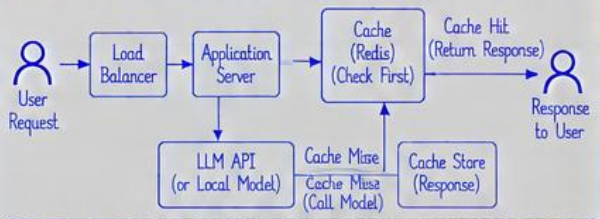
Metric	Description	Goal	How to Measure
Latency	Time to get response from model	Low (ms)	Response time, P95/P99
Throughput	Number of requests handled per second	High (RPS)	Requests per second
Cost	Total cost of using AI services	Low (₹/month)	Token usage, API bill
Accuracy	Quality of response (should not degrade)	High	Evaluation metrics
Resource Usage	CPU, Memory, GPU usage	Optimal	Monitoring tools

3 AI Application Request Flow (Without Optimization)



- Every request directly calls the model.
- No caching or batching.
- Higher latency and cost.
- Not optimized for high traffic.

4 Optimized Architecture with Caching & Scaling



- Check cache first (if same or similar query).
- If not in cache, call model and store response.
- Use load balancing and auto scaling for high traffic.
- Improves latency, reduces cost and increases throughput.

5 Caching Strategies

Strategy	Description	Use Case	Tools
Exact Match Caching	Cache full query and response	Repeated queries	Redis, Memcached
Semantic Caching	Cache based on embedding similarity	Similar queries	Redis + Vector DB (Qdrant, Pinecone)
TTL Based Caching	Cache with expiry time	Dynamic data	Redis (TTL)
Hierarchical Caching	In-memory + distributed cache	High traffic apps	Caffeine + Redis
Cache Invalidation	Remove outdated cache	Data updates	Manual / Event based

6 Reducing Latency Techniques

1. Use caching (exact or semantic)
2. Use streaming responses (token-by-token)
3. Keep prompts optimized (remove unnecessary context)
4. Use smaller and faster models when possible
5. Use local models for simple tasks
6. Use asynchronous processing for non-blocking requests
7. Use connection pooling and keep-alive
8. Optimize network latency (use nearer region/ CDN)

7 Increasing Throughput

1. Use batching (send multiple requests together)
2. Use asynchronous processing (async / non-blocking)
3. Use queue-based processing (Kafka, RabbitMQ, SQS)
4. Use multiple API keys and rate limit handling
5. Use load balancing across multiple model instances
6. Use horizontal scaling (more pods/instances)
7. Use lightweight models for simple tasks
8. Optimize request/response payload size

8 Batching and Concurrency

Batching Example (OpenAI API)

```
# Sending multiple prompts together
prompts = [
    "Summarize text 1",
    "Summarize text 2",
    "Summarize text 3"
]
response = client.responses.create(
    model="gpt-4o",
    input=prompts
)
```

Asynchronous Processing (Python)

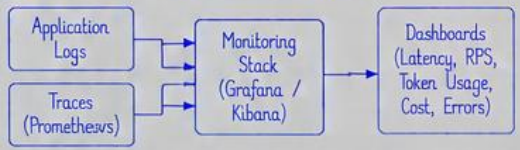
```
import asyncio
import aiohttp

async def call_model(prompt):
    async with aiohttp.ClientSession() as session:
        async with session.post(API_URL, json={
            "prompt": prompt
        }) as resp:
            return await resp.json()
```

9 Cost Optimization Techniques

Technique	Description	Benefit
Use Caching	Avoid repeated API calls	Reduce cost
Use Smaller Models	Use lightweight models (e.g., gpt-3.5)	Lower token cost
Prompt Optimization	Remove unnecessary tokens	Reduce token usage
Use Local Models	Deploy open-source models (Llama, Mistral)	No API cost
Batch Requests	Process multiple requests together	Lower cost per request
Monitor Token Usage	Track input/output tokens	Identify high usage
Use Structured Output	Short and concise response	Reduce output tokens

10 Monitoring and Profiling



- Monitor latency, throughput, error rate and token usage.
- Set up alerts for high latency or cost spikes.
- Use distributed tracing to find bottlenecks.
- Track model performance and resource usage (CPU/GPU).

11 Best Practices

1. Use caching for repeated and similar queries.
2. Optimize prompts and limit context size.
3. Use streaming for better user experience.
4. Implement batching and asynchronous processing.
5. Monitor performance, cost and errors regularly.
6. Use auto scaling for high traffic.
7. Choose the right model for your use case.
8. Keep security, accuracy and cost in balance.

12 Key Takeaways

- Performance optimization improves speed, user experience and reduces cost.
- Use caching, batching, async processing and scalable infrastructure.
- Monitor latency, throughput and token usage.
- Optimize prompts and choose the right model.
- Continuously analyze and improve based on real usage and metrics.
- A well-optimized AI application is faster, cheaper and more reliable.

By Rahul Yadav

16. Reliability, Fault Tolerance, and Resilience in AI Systems

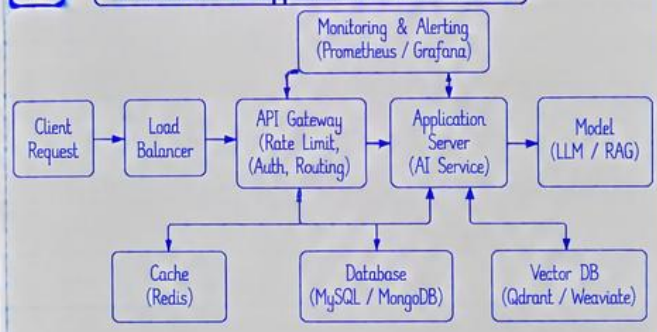
1 Overview

- Reliability and fault tolerance ensure AI applications provide consistent and correct responses, even when failures occur.
- AI systems depend on multiple components (models, APIs, databases, external tools), so failures can happen at different levels.
- This page covers common failures, resilience techniques, retry strategies, fallback mechanisms, and best practices to build reliable AI applications.

2 Common Failure Scenarios

Component	Failure Scenario	Example	Impact
Model	Model unavailable	LLM API down	No response
Network	High latency or timeout	Slow API/network	Increased latency
External API	Third-party API failure	Web search API down	Tool call fails
Database	Connection failure	DB not reachable	Data not found
Vector DB	Search service down	Qdrant/Weaviate down	RAG fails
Cache	Cache miss or failure	Redis unavailable	Higher latency
Infrastructure	Server or container crash	Pod/VM failure	Service downtime
Rate Limits	API quota exceeded	OpenAI rate limit	Requests rejected

3 Reliable AI Application Architecture



4 Fault Tolerance Techniques

1. Retry with exponential backoff for transient failures.
2. Use circuit breaker to stop calls to failing services.
3. Implement fallback mechanism (alternate model or cached response).
4. Use timeout for all external calls (model APIs, tools, DB).
5. Use bulkhead pattern to isolate failures.
6. Use graceful degradation (simpler response when resources low).
7. Use idempotency to handle duplicate requests.
8. Use health checks and auto-recovery.
9. Use multiple model providers for high availability.
10. Keep critical data in cache to reduce dependency on external services.

5 Retry and Backoff Strategy

- Retry for temporary failures (network error, 5xx, timeout).
- Use exponential backoff with jitter.
- Set maximum retry attempts.
- Do not retry for client errors (4xx).
- Example (Python - using tenacity):

```
from tenacity import retry, stop_after_attempt, wait_exponential
@retry(stop=stop_after_attempt(3),
       wait=wait_exponential(multiplier=1, min=2, max=10))
def call_llm(prompt):
    return client.chat.completions.create(...)
```

6 Circuit Breaker Pattern

- Prevents repeated calls to a failing service.
- Stops requests when failure rate is high.
- Allows recovery after a cooldown period.
- Common states: Closed → Open → Half-Open → Closed.



7 Fallback Strategies

Failure Type	Fallback Strategy	Example
Model API failure	Use another provider	OpenAI → Claude
High latency	Use cached response	Return previous answer
Tool/API failure	Skip tool call	Answer without tool
Vector DB down	Use keyword search	Traditional search
Low resources	Use smaller model	GPT-4 → GPT-3.5
Rate limit exceeded	Queue request	Process later
Complete failure	Return user-friendly message	"Service temporarily unavailable"

8 High Availability and Scalability

- Deploy multiple instances of AI service.
- Use load balancer to distribute traffic.
- Use auto scaling (e.g., Kubernetes HPA).
- Use stateless services where possible.
- Use distributed cache and database.
- Use multi-region deployment for critical applications.
- Implement health checks and readiness probes.
- Use blue-green or canary deployment for model updates.
- Keep model versions backward compatible.

9 Monitoring and Alerting

What to Monitor	Metrics / Tools	Why
API latency	Prometheus, Grafana	Detect slow responses
Error rate (4xx/5xx)	Prometheus, ELK	Identity failures
Model availability	Health checks	Ensure model is up
Token usage	Logs, dashboards	Track cost and usage
Queue size	RabbitMQ, Kafka	Detect backlog
Resource usage (CPU/GPU)	Kubernetes, Grafana	Avoid resource exhaustion
Custom business metrics	Application logs	Track user experience

10 Disaster Recovery and Resilience

1. Keep backups of important data (DB, Vector DB, configs).
2. Use multi-region or multi-cloud deployment.
3. Use infrastructure as code (Terraform, Helm).
4. Implement automated failover.
5. Regularly test disaster recovery plan.
6. Keep model and data versions.
7. Use data replication and snapshots.
8. Ensure quick rollback for failed deployments.
9. Maintain runbooks for common failure scenarios.
10. Communicate with users during service disruptions.

11 Common Challenges

1. Handling high latency and timeouts.
2. Managing model and API failures.
3. Dealing with rate limits and quotas.
4. Ensuring data consistency.
5. Handling large traffic spikes.
6. Maintaining reliability across multiple components.
7. Debugging errors in complex AI workflows.
8. Balancing cost, performance and availability.

10 Best Practices

1. Design for failure from the beginning.
2. Use retries, timeouts and circuit breakers.
3. Implement fallback and graceful degradation.
4. Monitor everything and set up alerts.
5. Use caching and asynchronous processing.
6. Keep your system stateless and scalable.
7. Test failure scenarios regularly.
8. Maintain good logging and tracing.
9. Use multiple model providers when possible.
10. Continuously review and improve reliability measures.

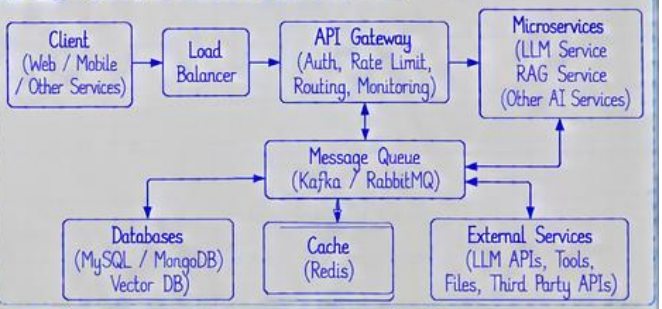
By Rahul Yadav

17. AI System Design – Microservices, APIs and Event-Driven Architecture

1 Overview

- Designing scalable and reliable AI applications requires well-structured architecture using microservices, APIs and event-driven systems.
- This page covers key components, architecture patterns, communication methods and best practices.
- We focus on real-world LLM/ML applications, but these principles apply to all AI systems.
- Goals: scalability, reliability, maintainability, cost efficiency and flexibility.

2 High Level Architecture



3 Microservices for AI Applications

- Break the application into small, independent services.
- Each service has a specific responsibility.
- Services can be developed, deployed and scaled independently.
- Common services in an AI application:

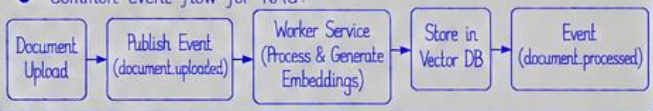
Service	Responsibility
API Gateway Service	Authentication, routing, rate limiting
User Service	User management, profiles, permissions
LLM Service	Interact with LLM (OpenAI, local model)
RAG Service	Document ingestion, retrieval, context building
Data Processing Service	Process, clean and chunk documents
Vector DB Service	Store and manage embeddings
Notification Service	Email / in-app notifications
Monitoring Service	Logs, metrics, alerts

4 Communication Patterns

Pattern	Description	Use Case	Example
Synchronous (REST API)	Direct request-response communication	Real-time inference	/generate /generateRAG
Asynchronous (Message Queue)	Event-based communication	Long running tasks	Kafka, RabbitMQ
gRPC	High performance binary communication	Internal service communication	Model service calls
WebSocket	Real-time bidirectional communication	Live chat, streaming response	Chat UI, streaming tokens

5 Event-Driven Architecture

- Services communicate using events (asynchronouss).
- Helps in decoupling services and improving scalability.
- Useful for long running AI tasks like document ingestion, embedding generation, batch processing, etc.
- Common event flow for RAG:

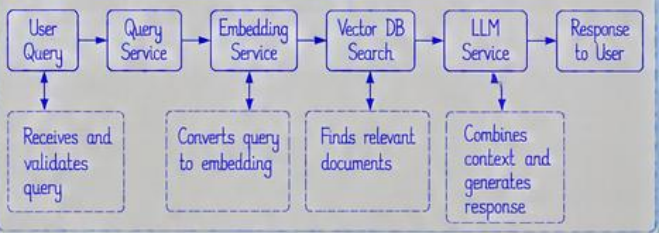


6 API Design for AI Services

- Use clear and consistent API design.
- Follow REST or gRPC based on requirement.
- Version your APIs (e.g., /v1/, /v2/).
- Use request and response schemas for validation.
- Handle errors with meaningful messages.

Endpoint	Method	Description
/generate	POST	Generate response using LLM
/generate-rag	POST	Generate response using RAG
/documents	POST	Upload and process document
/search	POST	Search relevant documents
/embeddings	POST	Generate embeddings
/health	GET	Service health check

7 Data Flow in RAG System (Microservices)



8 Scalability Strategies

1. Use containerization (Docker) and orchestration (Kubernetes).
2. Deploy services independently.
3. Use horizontal scaling (add more instances).
4. Use auto scaling based on CPU, memory or custom metrics.
5. Use caching to reduce repeated LLM calls.
6. Use asynchronous processing for heavy tasks.
7. Use load balancer and API gateway.
8. Use stateless services whenever possible.

9 Security Considerations

1. Use authentication and authorization (JWT, OAuth2).
2. Implement-role-based access control (RBAC).
3. Validate and sanitize all inputs.
4. Rate limiting to prevent abuse.
5. Encrypt sensitive data (in transit and at rest).
6. Use API keys for external LLM services.
7. Implement audit logs and monitoring.
8. Secure communication between microservices (mTLS).

10 Monitoring and Observability

Area	Tools	What to Monitor
Logs	ELK / Loki	Application logs, errors
Metrics	Prometheus + Grafana	Latency, throughput, resource usage
Tracing	Jaeger / OpenTelemetry	Request flow across services
Alerts	Grafana / AlertManager	Service downtime, high latency
Business Metrics	Custom Dashboard	Number of requests, token usage, cost

11 Best Practices

1. Keep services small and focused.
2. Use event-driven architecture for background tasks.
3. Implement retries, timeouts and circuit breakers.
4. Use caching to reduce latency and cost.
5. Monitor all services and set up alerts.
6. Version your models and APIs.
7. Keep the system secure and compliant.
8. Design for scalability, reliability and maintainability.

12 Key Takeaways

- Use microservices, APIs and event-driven architecture to build scalable and reliable AI applications.
- Decouple services using message queues.
- Follow security, monitoring and best practices.
- Design for real-world usage with scalability, fault tolerance and cost optimization.

By Rahul Yadav

18. AI Observability, Logging, Tracing, and Production Monitoring

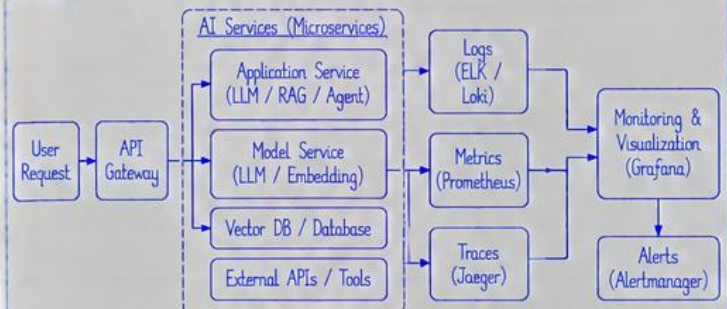
1 Overview

- Observability helps us understand the internal state of AI applications and debug issues in production.
- It includes logs, metrics, traces and alerts.
- AI applications have additional challenges like LLM latency, token usage, model errors and quality monitoring.
- This page covers tools, architecture, key metrics, implementation steps and best practices for observability in AI systems.

2 Why Observability is Important?

1. Find and fix issues quickly (errors, latency, timeouts).
2. Monitor model performance and quality.
3. Track resource usage (CPU, GPU, tokens, cost).
4. Understand user behavior and system health.
5. Detect anomalies and unusual patterns.
6. Ensure reliability, security and good user experience.

3 Observability Architecture for AI Applications



4 Key Components

Component	Purpose
Logging	Store application logs, errors and events.
Metrics	Collect numerical data (latency, rate, usage).
Tracing	Track request flow across services.
Visualization	Dashboards to analyze logs, metrics and traces.
Alerting	Notify when issues or anomalies occur.
AIOps	Use ML to detect anomalies and predict failures.

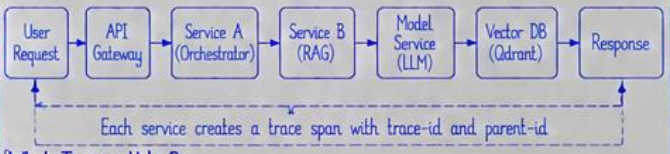
5 Types of Logs in AI Applications

Log Type	Description	Examples
Application Logs	General application logs	Request received, response sent.
Model Logs	Model related logs	Model name, version, input size, latency, token usage.
Error Logs	Exceptions and failures	Timeout, 5xx error, model failure.
Audit Logs	User and security events	User login, access, data upload.
Business Logs	Domain specific events	Document processed, query executed.

6 Important Metrics to Monitor

Metric Category	Metrics
Application Metrics	Request rate (RPS), error rate, latency (P50/P95/P99)
Model Metrics	Model latency, token usage, cost per request, model errors, success rate
RAG Metrics	Retrieval latency, number of chunks, relevance score, no. of sources used.
System Metrics	CPU, memory, GPU usage, disk usage, network I/O
Business Metrics	Active users, queries per user, feature usage

7 Distributed Tracing for AI Applications



What Tracing Helps?

1. See end-to-end request flow.
2. Identify which service is slow.
3. Find the exact bottleneck (DB, model, external API).
4. Analyze latency at each step.
5. Debug failures across multiple microservices.

8 Popular Tools

Tool	Type	Use Case
Prometheus	Metrics	Collect and store metrics.
Grafana	Visualization	Dashboards and alerting.
ELK Stack (Elasticsearch + Logstash + Kibana)	Logging	Centralized log management.
Loki	Logging	Lightweight log aggregation (for Kubernetes).
Jaeger	Tracing	Distributed tracing.
OpenTelemetry	Tracing + Metrics + Logs	Standard for observability.
Alertmanager	Alerting	Send alerts (email, Slack, etc).

9 Example: Logging in Python (FastAPI)

```
import logging
from fastapi import FastAPI

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - ((message)s"
)
logger = logging.getLogger(__name__)

app = FastAPI()

@ppp.get("/query")
async def query(q: str):
    logger.info(f"Received query: {q}")
    + process query
    try:
    ***   + call model
        | logger.info("Query processed succesfully")
        | return {"response": "success"}
    except Exception as e:
        logger.error(f"Error processing query: (str(e)}", exc_info=True)
        return {"error": "failed"}
```

10 Dashboards and Alerts

Common Dashboards

1. Request rate and error rate.
2. Latency (P50/P95/P99).
3. Model token usage and cost
4. GPU / CPU / Memory usage
5. RAG retrieval metrics
6. Active users and feature usage

Alert Examples

1. Error rate > 5%
2. Latency > threshold (e.g., 10s)
3. Model service down (no response)
4. High token usage (cost spike)
5. High CPU/GPU usage (> 80%)
6. Unusual drop in requests

11 Best Practices

1. Use structured logging (JSON format).
2. Include request-id / trace-id in all logs.
3. Monitor key metrics and set meaningful alerts.
4. Track model performance (latency, token usage, errors).
5. Use distributed tracing for multi-service architecture.
6. Keep logs for required period and manage storage cost.
7. Use dashboards for real-time monitoring.
8. Continuously analyze and improve based on observability data.

12 Key Takeaways

- Observability is essential for reliable and scalable AI applications.
- Use logs, metrics, traces and alerts together.
- Monitor both system-level and AI-specific metrics.
- Use the right tools (Prometheus, Grafana, ELK, Jaeger, OpenTelemetry).
- Set up meaningful dashboards and alerts.
- Follow best practices to quickly detect, troubleshoot and resolve issues.

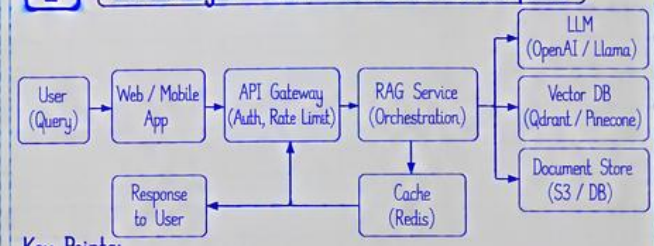
By Rahul Yadav

19. Real-World AI System Design Case Studies and Architecture

1 Overview

- In this page we will go through real-world case studies of AI applications and understand their system design.
- These examples show how different components (models, APIs, databases, vector stores, queues, monitoring, etc.) work together in production.
- We will cover common use cases like RAG system, chatbot, document processing, recommendation system, and multi-agent application.
- Each case study includes architecture, data flow and key design considerations.

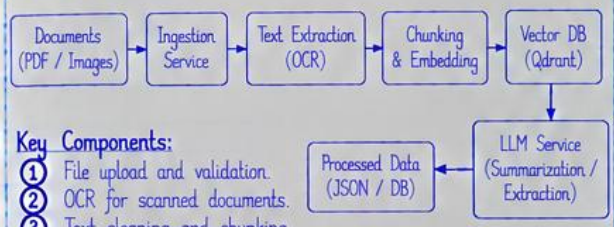
2 Case Study 1 - RAG based Chatbot (Enterprise)



Key Points:

1. Ingest documents (PDF, DOC, Web) and create embeddings.
2. Store embeddings in Vector DB.
3. Retrieve relevant chunks and send to LLM with context.
4. Use caching to improve latency and reduce cost.
5. Add monitoring, logging and feedback loop.

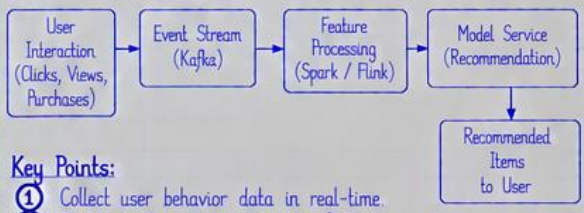
3 Case Study 2 - Document Processing System



Key Components:

1. File upload and validation.
2. OCR for scanned documents.
3. Text cleaning and chunking.
4. Generate embeddings and store in Vector DB.
5. Extract key information using LLM.
6. Store results in database and provide API to search.

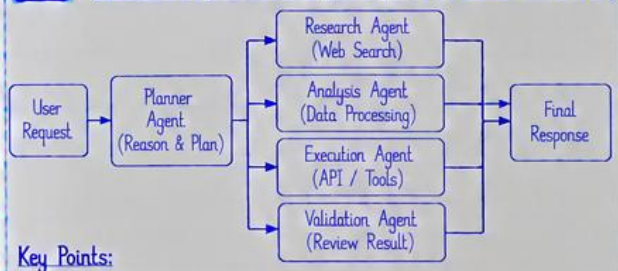
4 Case Study 3 - Recommendation System



Key Points:

1. Collect user behavior data in real-time.
2. Process and generate user/item features.
3. Use ML model for personalized recommendations.
4. Store user data and item embeddings.
5. Serve recommendations via low-latency API.
6. Continuously retrain model with new data.

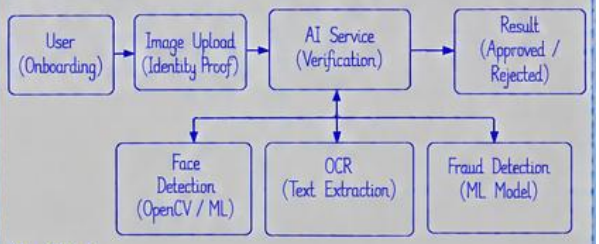
5 Case Study 4 - Multi-Agent System (Task Automation)



Key Points:

1. Use multiple specialized agents for complex tasks.
2. Each agent has a specific role (research, analysis, execution, validation).
3. Agents can use tools (search, APIs, database, files).
4. Use memory to maintain context between agents.
5. Implement guardrails and human-in-the-loop for critical tasks.

6 Case Study 5 - AI in Banking (Example: KYC / Fraud)



Key Points:

1. Capture user identity documents and selfie.
2. Use ML models for face match and document verification.
3. Extract text using OCR and validate with database.
4. Check for fraud using ML model and business rules.
5. Provide final decision with audit logs.

7 Comparison of Case Studies

Use Case	Main Components	Key Technologies	Key Challenges
RAG Chatbot	LLM, Vector DB, Cache	OpenAI, Qdrant, Redis	Latency, cost, relevant results
Document Processing	OCR, LLM, Vector DB	Tesseract, OpenAI, Qdrant	Accurate extraction, large files
Recommendation	Event Stream, ML Model, DB	Kafka, Spark, ML	Real-time processing, scalability
Multi-Agent System	Multiple Agents, Tools, Memory	LangChain, OpenAI, APIs	Task coordination, reliability
Banking (KYC/Fraud)	Computer Vision, OCR, ML	OpenCV, ML Models	Accuracy, fraud prevention, security

8 Design Considerations

1. Choose the right architecture (monolith vs microservices).
2. Ensure scalability and high availability.
3. Use caching to reduce latency and cost.
4. Implement proper authentication and security.
5. Monitor logs, metrics and traces.
6. Handle failures with retry, fallback and circuit breakers.
7. Use event-driven architecture for asynchronous tasks.
8. Optimize for cost, performance and accuracy.
9. Keep data privacy and compliance in mind.
10. Continuously evaluate and improve the system.

9 Best Practices

1. Start with a clear problem statement and requirements.
2. Design modular and scalable architecture.
3. Use managed services where possible (e.g., OpenAI, Qdrant, Kafka).
4. Implement proper monitoring and alerting.
5. Optimize for latency, cost and user experience.
6. Use feedback loop to improve model performance.
7. Follow security and compliance best practices.
8. Document the architecture and maintain runbooks.

10 Key Takeaways

- Real-world AI applications require combination of multiple components.
- System design should consider scalability, reliability, cost and security.
- RAG, document processing, recommendation and multi-agent systems have different architectures but follow similar principles.
- Monitoring, caching, event-driven design and best practices are critical for production success.
- Start simple, iterate and continuously improve based on real usage.

By Rahul Yadav

20. Complete AI System Design Interview Guide
Frameworks, Trade-offs, and Final Checklist

1 Overview

- This final page provides a complete guide to crack AI system design interviews.
- It includes step-by-step frameworks, important design considerations, trade-offs, common questions and a final checklist.
- Use this as a quick reference before interviews.
- Focus on clear communication, diagrams, trade-offs and real-world experience.

2 Step-by-Step System Design Framework

1. Understand requirements (functional + non-functional).
2. Clarify scope and ask questions.
3. Define high level architecture.
4. Identify key components and their responsibilities.
5. Design data flow and APIs.
6. Discuss scaling, reliability, security and cost.
7. Mention trade-offs and alternatives.
8. Consider monitoring, evaluation and continuous improvement.
9. Summarize the design and take questions.

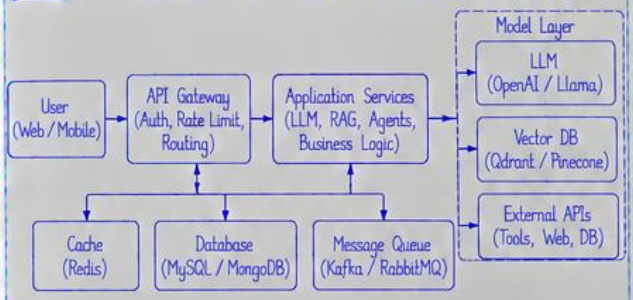
3 Common AI System Design Questions

1. Design a RAG based chatbot for enterprise data.
2. Design a document processing system (PDF, images, OCR).
3. Design a recommendation system.
4. Design a multi-agent system for task automation.
5. Design an AI system for fraud detection (KYC/fraud).
6. Design a real-time AI application (voice, video, chat).
7. Design an LLM fine-tuning and model serving platform.
8. Design a scalable vector database solution.
9. How will you monitor and evaluate LLM applications?
10. How will you reduce cost and latency for AI systems?

4 Key Design Considerations

Consideration	Key Points
Scalability	Handle high traffic, horizontal scaling, auto scaling
Reliability	Retry, fallback, circuit breaker, high availability
Latency	Caching, smaller models, async processing, closer region
Cost	Optimize model usage, caching, batching, right model
Security	Authentication, authorization, data encryption, PII masking
Data Privacy	Anonymization, compliance (GDPR, RBI, HIPAA)
Model Selection	Right model for use case (GPT, Llama, Mistral, etc.)
Evaluation	Accuracy, relevance, hallucination, user feedback
Maintainability	Modular design, microservices, good documentation
User Experience	Fast, accurate, safe and helpful responses

5 Generic AI System Architecture (Reference)



6 Trade-offs in AI System Design

Decision	Option 1	Option 2	Trade-off
Model Type	GPT-4	Llama / Mistral	High accuracy vs Lower cost & control
Vector DB	Pinecone	Qdrant / Weaviate	Managed service vs Self-hosted & flexible
Architecture	Monolith	Microservices	Simple vs Scalable and maintainable
Cache	No cache	Redis cache	Higher latency vs Lower latency & cost
Processing	Synchronous	Asynchronous	Simple vs Better scalability
Deployment	Single region	Multi region	Lower cost vs High availability
Model Hosting	API (OpenAI)	Self-hosted	Easy to use vs More control, infra cost

7 Example Interview Answer Structure

1. Problem Understanding → Repeat the problem in your own words.
2. Requirements → Functional and non-functional.
3. High Level Design → Draw architecture diagram.
4. Component Design → Explain each component.
5. Data Flow → Step-by-step flow.
6. Scalability & Reliability → How to handle high load and failures.
7. Security & Compliance → Auth, data privacy, PII, etc.
8. Trade-offs → Explain why you chose this approach.
9. Monitoring & Evaluation → Logs, metrics, feedback, model eval.
10. Test Scenarios → How to validate the system.

8 Important Frameworks and Concepts to Mention

Framework / Concept	Use Case
RAG (Retrieval Augmented Generation)	Enterprise chatbot, document Q&A
LangChain / LlamaIndex	Build LLM applications
Vector Databases (Qdrant, Pinecone)	Store and search embeddings
Kafka / RabbitMQ	Asynchronous tasks, event-driven systems
Kubernetes	Container orchestration and scaling
Prometheus + Grafana	Monitoring and observability
OpenTelemetry + Jaeger	Distributed tracing
Redis	Caching and session storage
Docker	Containerization
CI/CD (GitHub Actions, Jenkins)	Automated deployment
Model Evaluation (Ragas, LangSmith)	Evaluate LLM responses
Guardrails (NeMo Guardrails, Rebuff)	Safety and content filtering

9 Common Mistakes to Avoid

1. Not understanding the problem properly.
2. Jumping to solution without discussing requirements.
3. Ignoring scalability, reliability and security.
4. Not considering cost and latency.
5. Overcomplicating the design.
6. Missing monitoring and evaluation.
7. Not discussing trade-offs.
8. Forgetting real-world constraints and business needs.

10 Final Checklist Before Interview

- ☐ Understand common AI system design questions.
- ☐ Practice drawing clean architecture diagrams.
- ☐ Be ready with real-world examples from your experience.
- ☐ Know key tools and frameworks (RAG, LangChain, Kafka, etc.).
- ☐ Understand trade-offs and alternatives.
- ☐ Be prepared to discuss scalability, cost, security and monitoring.
- ☐ Practice explaining your approach clearly and confidently.
- ☐ Keep learning and stay updated with latest trends.

11 Key Takeaways

1. There is no single perfect design — discuss trade-offs.
2. Keep the design simple, scalable and production ready.
3. Focus on real-world use cases and practical constraints.
4. Use the right tools and frameworks for the problem.
5. Always consider latency, cost, security and reliability.
6. Monitoring, evaluation and feedback loop are essential.
7. Clear communication and structured approach are key.
8. Practice regularly with different case studies.
9. Stay updated with latest AI technologies and best practices.
10. Be confident and explain your thought process.

By Rahul Yadav